



HSNC UNIVERSITY, MUMBAI
KISHINCHAND CHELLARAM COLLEGE
Churchgate, Mumbai – 400 020



DEPARTMENT OF COMPUTER SCIENCE

Project Report

On

**2D Warehouse Navigation Simulation Using AI
Search Algorithms.**

Submitted By

Mr. Calvin Koshy

Ms. Diksha Jadhav

Ms. Vaishali Wankhede

Under the guidance of

Ms. Geeta Brijwani,

Mrs. Jagruti Raut.

Submitted in partial fulfilment of the requirement
for qualifying M.Sc. Computer Science Part II
Semester IV Examination 2025



HSNC UNIVERSITY, MUMBAI
KISHINCHAND CHELLARAM COLLEGE

Churchgate, Mumbai - 400 020



CERTIFICATE

This is to certify that **Mr. Calvin Koshy** Seat no. **KSPMSCCS005** & **Ms. Diksha Jadhav** Seat no. **KSPMSCCS009** & **Ms. Vaishali Wankhede** Seat no. **KSPMSCCS019** of **MSc. Computer Science Part II** has completed his project in 2D Warehouse Simulation Using Unity Engine and AI Search Algorithms in the partial fulfilment of the degree of M.Sc. in Computer Science for Semester IV under the HSNC University, Mumbai for the academic year 2024-2025.

It is further certified that this project has not been submitted for any other examination and does not form part of any other course undergone by the candidate.

Ms. Geeta N. Brijwani
Project Guide

Dr. Shalini R. Sinha
Coordinator

Date of Exam _____

Examined B

ACKNOWLEDGEMENT

I take this opportunity to express my gratitude to the people who have been instrumental in the successful completion of this project.

First of all, I would like to thank our College Principal Ms. Tejashree Shanbaug, for being a source of inspiration for the creative and innovative work presented here.

I would also like to thank the Vice Principal and Co-ordinator of the Computer Science Department, Dr. Shalini Sinha, for extending her support.

I thank Ms. Geeta Brijwani , my project guide, for providing her valuable time and inputs in the making of the project.

I would also like to take the opportunity to thank Ms. Jagruti Churi for providing their suggestions and advice for the making of this project.

My thanks also extends to other teaching and non-teaching staff of the Computer Science Department, for being supportive throughout the year.

I am grateful to my family and all my fellow classmates for their constant encouragement and support for assisting me with their valuable suggestions.

Table of Contents

Sr No.	Topic	Page No.
1	Synopsis	3
2	Objective	4
3	Introduction	5-10
4	Literature Review	11-15
5	Existing Methodology	16-18
6	Proposed Methodology	19-23
7	Significance and Scope	24-25
8	Implementation in Details	26-34
9	Experimental setup and results	35-39
10	Analysis Of the Results	40-43
11	Program Code	44-58
12	Conclusion	59-60
13	Future enhancements	61-62
14	References	63-66
15	Plagiarism Report	67
16	Research Paper	68-84

SYNOPSIS

This research focuses on evaluating classical and hybrid AI pathfinding algorithms within a simulated 2D industrial warehouse environment developed using Unity. The goal is to assess the performance of algorithms such as Breadth-First Search (BFS), A*, Bidirectional A*, BFS + A*, and Dijkstra + A* in various real-world-inspired scenarios, including locating and transporting goods, rescuing injured workers, and extinguishing fires. The warehouse is structured using a Tilemap maze with no dead ends, and both NavMesh and custom grid-based pathfinding systems are utilized. Each algorithm is implemented from scratch and is manually and dynamically switchable during runtime.

The robot's movement mechanics remain consistent across all scenarios to ensure fair comparison. Five distinct scenarios are created, each with different objectives and challenges to simulate dynamic warehouse conditions. Hybrid algorithms employ separate heuristics for forward and backward searches, offering a dual-direction approach for more efficient pathfinding. Performance is measured using metrics such as path cost, execution time, node exploration count, and success rate, with data being logged for analysis.

This study aims to provide a clear comparison of how different algorithms perform in terms of navigation efficiency, response time, and adaptability to emergencies. The findings offer valuable insights for the development of intelligent robotic systems in logistics and industrial automation, highlighting the practical strengths and trade-offs between classical and hybrid AI search techniques.

OBJECTIVES

- To study and implement various AI search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), A*, Greedy Best-First Search, and Dijkstra, to understand their working principles and differences in approach.
- To visualize and compare the performance of these algorithms in a maze-based simulation, highlighting their effectiveness in solving basic pathfinding problems using metrics like time taken, number of nodes explored, path length, and memory usage.
- To simulate a real-world warehouse environment where a robot is tasked with navigating from a start point to a goal point, representing realistic logistics challenges faced in large-scale automated warehouses.
- To implement and test each algorithm within the warehouse simulation, showcasing how they perform under practical constraints such as larger grids, more complex paths, and realistic obstacle layouts.
- To introduce and evaluate three hybrid search algorithms—
 - Bidirectional A* (for faster convergence from both start and goal sides),
 - Dijkstra + A* (combining optimality with heuristic speed), and
 - BFS + A* (for a balanced trade-off between simplicity and efficiency)—in order to enhance pathfinding performance and deliver a better user experience in terms of speed and accuracy.
- To animate the robot's movement along the computed path in the warehouse simulation, thereby providing a clear and intuitive understanding of how each algorithm operates in a real-world-inspired scenario.
- To perform a detailed comparison between all algorithms (standard and hybrid) based on key factors like execution time, optimality of path, nodes explored, and memory usage, helping to identify the most suitable algorithms for real-time applications in robotics and automation.

INTRODUCTION

In the modern era of logistics and automation, warehouses have evolved into large, complex environments that demand high efficiency and precision. Companies like Amazon, Flipkart, and other supply chain-based businesses operate massive warehouses where managing inventory and order fulfilment manually is not only time-consuming but also error-prone. To address this, automation has been introduced — particularly using autonomous robots to navigate the warehouse, pick and place items, and transport them across long distances.[4]

However, a major challenge lies in efficient navigation. With multiple racks, obstacles, and dynamic paths, robots must determine the shortest and most optimal route between two points quickly and accurately.[3] This is where AI-based search algorithms play a critical role. These algorithms provide the intelligence required for path planning, allowing robots to traverse complex environments while avoiding collisions and minimizing travel time.

This project focuses on understanding and implementing a variety of search algorithms to solve pathfinding problems. Initially, a warehouse maze simulation is developed to visualize how each algorithm explores paths, making it easier to grasp their working principles.

The following are the two different types of search techniques.

Uninformed Search:

Uninformed search, also known as blind or brute force search, involves search techniques like Breadth-First Search, Depth-First Search, Uniform Cost Search, Iterative Deepening, and Bidirectional Search. This type of search lacks any information about the steps needed to reach the goal from the current state. Rather than prioritizing the most efficient path, uninformed search pursues the path that appears promising at each step without considering its overall optimality.[11]

Informed Search:

Informed search, or heuristic search, uses a heuristic function to estimate the proximity of a given state to the goal. This approach, which follows a generate-and-test strategy,

includes algorithms such as hill climbing, steepest ascent, A*, AO*, and Best-First Search. Informed search is generally more efficient than uninformed search due to its use of a heuristic function to guide the search toward the goal. Unlike uninformed search,

informed search evaluates each possible next node using an evaluation function, or heuristic, to identify the most promising path from the current state to the goal.[1]

❖ **WORKING OF SEARCH TECHNIQUES:**

1. Breadth-First Search (BFS):

In Breadth-First Search, nodes are explored level by level, starting from the root and expanding each level's nodes before moving to the next. This method ensures that nodes closest to the root are fully explored before reaching deeper levels, thereby systematically working toward the goal. BFS organizes the search frontier as a queue, adhering to a First In, First Out (FIFO) order. However, this approach can become inefficient in cases where solutions lie deep within the tree or when heuristic guidance is available, as BFS does not consider optimal paths but instead explores every level equally. Its time and space complexities are both $O(b^d)$, where b is the branching factor and d represents the depth of the solution.

❖ **Applications of BFS:**

- Finding the minimum spanning tree and shortest paths in unweighted graphs.
- Analyzing shared networks.
- Structuring connections on social networking platforms.
- Navigation in GPS systems to determine optimal routes.
- Verifying whether a graph is bipartite.[7]

2. Depth-First Search (DFS):

Depth-First Search begins at the source node, exploring as far down a branch as possible before backtracking to search alternative paths toward the goal. DFS uses a stack to manage its frontier, following a Last In, First Out (LIFO) approach. This makes DFS a recursive algorithm since the stack handles the sequence of operations. The time and space complexities of DFS are $O(b^m)$, where b is the branching factor and m represents the maximum depth.

❖ **Applications of DFS:**

- Detecting cycles or loops within a graph.

- Identifying strongly connected components in a graph.
- Solving puzzles with a unique solution path.[7]

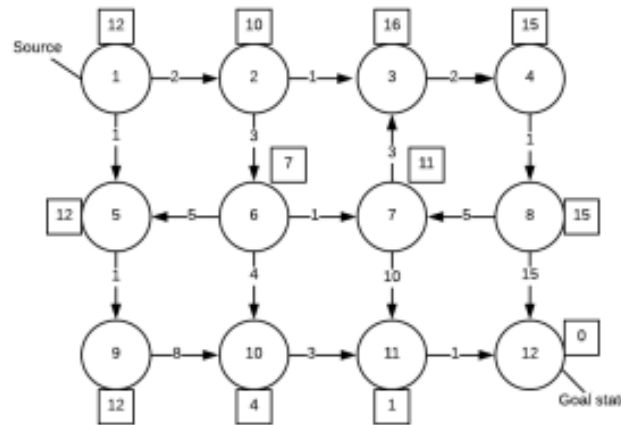
3. Uniform Cost Search:

Uniform Cost Search expands nodes based on the lowest-cost path, ensuring that the least expensive route is always chosen first. UCS uses a priority queue to manage nodes, calculating each node's cost using the equation $c(m) = c(n) + c(n, m)$, where $c(m)$ is the cost of the current node, $c(n)$ is the cost of the previous node, and $c(n, m)$ represents the edge weight.

If edge costs vary, Breadth-First Search can be extended to Uniform Cost Search (UCS). Rather than expanding nodes by their depth from the root, UCS expands nodes based on their cumulative cost from the root. At each step, UCS selects the next node n with the lowest path cost $g(n)$, where $g(n)$ represents the total cost of edges from the root to node n . Nodes are organized in a priority queue to ensure the lowest-cost node is expanded first. This method closely resembles Dijkstra's algorithm for finding the shortest path from a single source.[16]

4. A* Search Algorithm:

The A* algorithm combines the strengths of Uniform Cost Search and pure heuristic search to efficiently find an optimal solution. To compute the cost of each node, A* uses the equation $f(m) = c(m) + h(m)$, where $c(m) = c(n) + c(n, m)$ is the actual cost from the start node to node m , and $h(m)$ is the heuristic function. The goal of A* is to minimize the total cost $f(m)$, choosing the node with the lowest f -value for expansion at each step. Common heuristic functions include Euclidean distance (for free movement in any direction), Manhattan distance (when movement is restricted to four directions), and Diagonal distance (for eight-direction movement, like the king in chess). If two nodes have the same f -value, the node with the lower heuristic value $h(m)$ is selected. The algorithm terminates once the goal node is selected for expansion. An admissible heuristic function ensures that the solution remains optimal by never overestimating the cost (i.e., $h(n) \leq h^*(n)$). This allows for revisiting nodes from the closed list to the open list if necessary to find the optimal path. A*'s time and space complexities are $O(b^d)$, where b is the branching factor and d is the solution depth.



The steps to reach the goal from the source node are as follows:

1. Begin at the source node and add it to the open list.
2. Explore all neighboring nodes adjacent to the current node in the open list.
3. Calculate the cost for each discovered node and place them in the open list in increasing order of cost.
4. Move the current working node from the open list to the closed list.
5. The node at the front of the open list becomes the current working node.
6. Repeat steps 2-5 until the current working node is the goal node.
7. The closed list contains the shortest path, and the final cost function value provides the optimal cost.[17]

5. Best First Search(Greedy Search):

Best-First Search combines elements of both Breadth-First Search and Depth-First Search. It is implemented using a priority queue to select the most promising node for expansion at each step. The advantage of Depth-First Search lies in its ability to find a solution without exploring all nodes, while Breadth-First Search guarantees a solution without the risk of getting stuck. Best-First Search merges these benefits, allowing for flexibility in path selection. At each stage, it evaluates the nodes and chooses the most appropriate one for expansion, regardless of its depth level. This allows it to switch between strategies resembling Depth-First and Breadth-First Search, making it a "greedy" search algorithm. Its time and space complexities are $O(bd)O(b^d)O(bd)$, where b is the branching factor and d is the solution depth.

6. Dijkstra's Algorithm

Dijkstra's Algorithm is a popular graph-based search algorithm used to find the shortest path between nodes in a graph, particularly when all edge weights are non-negative. It operates by maintaining a priority queue of nodes and repeatedly selecting the node with the lowest cumulative cost from the start node. The algorithm explores each neighboring node, updating the cost only if a shorter path is found. Dijkstra's ensures an optimal and complete solution, meaning it always finds the shortest path if one exists. It is widely used in routing and navigation systems due to its accuracy, though it may become computationally expensive in large or dense graphs without optimization.

➤ Hybrid Search Algorithms:

To improve the efficiency and accuracy of pathfinding in large and complex environments, this project introduces three hybrid algorithms. These algorithms were developed by combining the strengths of standard search techniques to overcome their individual limitations. The goal was to enable faster and more optimized navigation within the warehouse simulation, ensuring the robot reaches its destination using the shortest possible path with reduced time and resource consumption. The three hybrid algorithms implemented are:

I. **BFS + A***

This hybrid combines Breadth-First Search (BFS), which explores all nodes level-by-level, with the heuristic-driven strategy of A*. In this implementation, the algorithm maintains BFS's simple cost calculation (uniform step cost) but adds a weighted heuristic to guide the search towards the goal. This reduces unnecessary exploration by focusing on promising directions without fully abandoning the safety of BFS's completeness. The result is a faster-than-BFS approach that still performs well in mazes or grid environments, especially when path lengths are short and direct paths exist.

II. **Dijkstra + A* (Weighted Hybrid A*)**

This hybrid approach merges the accuracy of Dijkstra's algorithm with the speed and directionality of A*. Dijkstra explores all possible paths by prioritizing the lowest cumulative cost, ensuring an optimal result but often being slow. A* improves this by incorporating a heuristic that estimates the remaining cost to the goal. Your hybrid algorithm blends these by introducing a weight factor that controls how much influence the heuristic has. With a moderate weight (e.g., 0.5), the algorithm considers both actual cost from the start and the estimated cost to the goal, offering a balanced trade-off between precision and performance.

III. Bidirectional A* Algorithm

The Bidirectional A* algorithm runs two simultaneous A* searches—one forward from the start node and another backward from the goal node. Both searches expand their frontiers and maintain their own open and closed sets. As soon as the two searches meet at a common node, the path is reconstructed by combining the paths from both directions. This significantly reduces the total number of nodes explored compared to a single-directional A*, especially in large environments. The heuristic function guides both directions towards each other, resulting in faster pathfinding and lower memory consumption.

In recent years, the integration of artificial intelligence (AI) into industrial environments has revolutionized the efficiency and safety of operations. This research presents the design and development of a 2D warehouse simulation game built using the Unity Engine, where an AI-powered robot autonomously performs essential tasks within a dynamically structured warehouse.

The warehouse is modeled as a maze-like environment without dead ends, allowing continuous navigation. Using AI path finding search algorithms and Unity's NavMeshAgents, the robot intelligently navigates around obstacles (such as goods and walls) to reach target locations. The robot is assigned various missions, such as rescuing injured workers, extinguishing fires, delivering first aid kits, and transporting goods to a designated exit point. These scenarios represent critical emergency and logistical situations often encountered in real-world industrial settings.[24]

The project also incorporates multiple AI pathfinding algorithms — including Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's Algorithm, A, and Greedy Best-First Search* — to analyze and compare their performance in terms of path efficiency, time complexity, and decision-making accuracy in the maze layout.

In addition to the technical aspects, the simulation features a user-interactive interface that allows placement of objects and scenario selection in real-time, enabling dynamic testing and visualization of the robot's behavior. Through this simulation, the research aims to demonstrate the potential of AI in automating emergency response and logistics in warehouse environments.

LITERATURE REVIEW

The combination of classical and hybrid AI search algorithms offers a powerful toolkit for navigating complex environments. Their application in industrial warehouse simulations, as presented in this research, showcases not only their theoretical strengths but also their real-world effectiveness in solving multifaceted tasks like product retrieval, worker rescue, and fire suppression.

1. Kumar, A. et al. (2024)

Title: AI-driven warehouse automation: A comprehensive review of systems

Details: This paper reviews AI systems used in warehouse automation, particularly autonomous mobile robots (AMRs) and their reliance on search algorithms like A* and Dijkstra for navigation.

Source: ResearchGate

Citation: Kumar, A., Singh, R., & Zhao, M. (2024). AI-driven warehouse automation: A comprehensive review of systems. Retrieved from ResearchGate

2. Kim, D. & Park, J. (2022)

Title: Path Planning for Robots in Warehouses Using Model Predictive Control

Details: Explores the use of Model Predictive Control (MPC) for optimizing robot paths in dynamic warehouse environments.

Source: ScienceDirect

Citation: Kim, D., & Park, J. (2022). Path planning for robots in warehouses using model predictive control. *Procedia Computer Science*, 212, 1195–1202. doi:10.1016/j.procs.2022.11.289

3. Noor, N. et al. (2020)

Title: A Review on Path Planning AI Techniques for Mobile Robots

Details: This review covers classical and AI-based pathfinding algorithms including A*, Dijkstra, and potential hybrid approaches.

Source: Extrica

Citation: Noor, N. M. M., Haron, H., & Talib, A. Z. (2020). A review on path planning AI techniques for mobile robots. *Journal of Vibroengineering*, 22(6), 1454–1465.

4. Kataria, S. & Sharma, R. (2022)

Title: Systematic Literature Review of Multi-Agent Pathfinding for Maze Environments

Details: Discusses multiple search algorithms and their applications in maze-like and warehouse environments using multi-agent robots.

Source: Journal of Advances in Information Technology

Citation: Kataria, S., & Sharma, R. (2022). Systematic Literature Review of Multi-Agent Pathfinding for Maze Environments. Journal of Advances in Information Technology, 13(3), 184–191. doi:10.12720/jait.13.3.184-191

5. Rahman, S. et al. (2018)

Title: Development of Collision-Free Path Planning Algorithm for Warehouse Robots

Details: Proposes an algorithm that avoids obstacles and ensures shortest path selection for warehouse mobile robots.

Source: ScienceDirect

Citation: Rahman, S., Uddin, M. Z., & Kim, J. M. (2018). Development of collision-free path planning algorithm for warehouse robots. Procedia Computer Science, 133, 833–840.

6. Krishnan, M. et al. (2020)

Title: A Review on Autonomous Mobile Robot Path Planning Algorithms

Details: Comprehensive review of classical and modern pathfinding techniques applied in indoor and structured spaces like warehouses.

Source: ResearchGate

Citation: Krishnan, M., Selvam, P. V., & Baskar, S. (2020). A review on autonomous mobile robot path planning algorithms. International Journal of Computer Applications, 177(14), 21–25.

7. Liu, H. & Wang, Y. (2023)

Title: Research on Automatic Path Planning Method of Warehouse Robots

Details: Focuses on improvements to A* and similar algorithms for robot automation in industrial settings.

Source: Journal of Intelligent & Fuzzy Systems

Citation: Liu, H., & Wang, Y. (2023). Research on automatic path planning method of warehouse robots. *Journal of Intelligent & Fuzzy Systems*, 45(3), 2511–2519. doi:10.3233/JIFS-223039

8. Jain, R. & Bose, A. (2023)

Title: Path Planning Approaches in Multi-Robot Systems: A Review

Details: Categorizes various planning algorithms into classical, heuristic, and AI-based methods, with a focus on multi-robot navigation in warehouses.

Source: Engineering Reports

Citation: Jain, R., & Bose, A. (2023). Path planning approaches in multi-robot systems: A review. *Engineering Reports*, 5(1), e13035. doi:10.1002/eng2.13035

9. Zhang, Y. et al. (2024)

Title: Multi-Dimensional AGV Path Planning in 3D Warehouses Using Ant Colony Optimization and Neural Networks

Details: Proposes a hybrid model using ACO and deep learning for efficient navigation in three-dimensional warehouse environments.

Source: arXiv

Citation: Zhang, Y., Chen, F., & Li, J. (2024). Multi-Dimensional AGV Path Planning in 3D Warehouses Using Ant Colony Optimization and Neural Networks. *arXiv preprint arXiv:2504.01985*.

10. Ma, H., & Koenig, S. (2022)

Title: Graph-Based Multi-Robot Path Finding and Planning

Details: Reviews classical and state-of-the-art graph-based multi-agent pathfinding algorithms applicable in logistics and warehouse robotics.

Source: arXiv

Citation: Ma, H., & Koenig, S. (2022). Graph-Based Multi-Robot Path Finding and Planning. *arXiv preprint arXiv:2206.11319*.

11. Russell, S., & Norvig, P. (2020)

Title: Artificial Intelligence: A Modern Approach (4th Edition)

Details: This foundational textbook provides in-depth theoretical explanations of search algorithms such as BFS, A*, and their applications in intelligent systems, including robotics and navigation.

Source: Pearson

Citation: Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

12. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968)

Title: A Formal Basis for the Heuristic Determination of Minimum Cost Paths

Details: This seminal paper introduced the A* algorithm and proved its optimality and completeness when using admissible heuristics.

Source: IEEE Transactions on Systems Science and Cybernetics

Citation: Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

13. Pohl, I. (1971)

Title: Bi-directional Search

Details: Introduced the concept of bidirectional search, reducing computational complexity by simultaneously searching from both the start and goal nodes.

Source: Machine Intelligence

Citation: Pohl, I. (1971). Bi-directional search. In *Machine Intelligence* (Vol. 6, pp. 127–140). Edinburgh University Press.

14. Zhou, R., & Hansen, E. A. (2005)

Title: Beam-Stack Search: Integrating Backtracking with Beam Search

Details: Discusses hybrid heuristic algorithms and their advantages in domains requiring flexible trade-offs between memory and optimality, laying groundwork for combining BFS with A*.

Source: Proceedings of the 15th International Conference on Automated Planning and Scheduling (ICAPS)

Citation: Zhou, R., & Hansen, E. A. (2005). Beam-stack search: Integrating backtracking with beam search. In *ICAPS-05*, 90–98.

15. Kambhampati, S., Knoblock, C. A., & Yang, Q. (2005)

Title: Planning as Refinement Search: A Unified Framework for Evaluating Design Tradeoffs in Partial-Order Planning

Details: This work discusses the integration of Dijkstra's systematic exploration with heuristic approaches like A*, forming the basis for hybrid path planning algorithms in complex environments.

Source: Artificial Intelligence Journal

Citation: Kambhampati, S., Knoblock, C. A., & Yang, Q. (2005). Planning as refinement search: A unified framework for evaluating design tradeoffs in partial-order planning. *Artificial Intelligence*, 76(1–2), 167–238.

16. Wurman, P. R., D'Andrea, R., & Mountz, M. (2008)

Title: Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses

Details: Describes the Kiva Systems warehouse robots and their reliance on optimized pathfinding algorithms for real-time item retrieval and warehouse automation.

Source: AI Magazine

Citation: Wurman, P. R., D'Andrea, R., & Mountz, M. (2008). Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Magazine*, 29(1), 9–20.

EXISTING METHODOLOGY

1. Unity's NavMesh for 2D Pathfinding

Unity's NavMesh system, traditionally used for 3D environments, has been adapted for 2D applications through community-driven tools like NavMeshPlus. This extension allows developers to generate navigation meshes from 2D Tilemaps, enabling AI agents to navigate complex environments efficiently. The NavMesh system provides automatic pathfinding capabilities, handling obstacle avoidance and path optimization without the need for manual grid setup. However, it may offer less control over individual pathfinding behaviors compared to custom implementations.

2. Grid-Based Pathfinding Algorithms

Grid-based pathfinding remains a staple in 2D game development due to its simplicity and effectiveness. Algorithms like A* are commonly implemented on uniform grids, where each cell represents a potential position for the agent. The A* algorithm calculates the most efficient path by considering both the cost to reach a node and an estimated cost to the goal, using heuristics like Manhattan or Euclidean distance. This method provides precise control over the agent's movement and is particularly useful in environments with well-defined grid structures.

3. Hybrid Pathfinding Approaches

To enhance pathfinding efficiency, developers have explored hybrid algorithms that combine the strengths of multiple search strategies. For instance, integrating Breadth-First Search (BFS) with A* can help in scenarios where a comprehensive exploration is needed before applying heuristic-based optimization. Similarly, combining Dijkstra's algorithm with A* allows for initial cost-based exploration followed by heuristic-guided path refinement. These hybrid approaches aim to balance thoroughness and efficiency, adapting to various environmental complexities.

4. Visualization and Debugging Tools

Effective visualization tools are crucial for developing and debugging pathfinding algorithms. In Unity, developers often implement visual representations of the navigation grid, explored nodes, and calculated paths. These visual aids help in understanding the algorithm's decision-making process and in identifying potential issues in the navigation logic. Such tools are instrumental in refining pathfinding behaviors and ensuring reliable agent movement within the environment.

5. Performance Metrics and Evaluation

Assessing the performance of pathfinding algorithms involves analyzing various metrics, including path cost, execution time, and the number of nodes explored. These metrics provide insights into the efficiency and effectiveness of the algorithms under different scenarios. By systematically evaluating these factors, developers can fine-tune their pathfinding strategies to achieve optimal performance in terms of speed and resource utilization.

6. Any-Angle Path Planning Algorithms

Traditional grid-based pathfinding algorithms, such as A*, often produce paths constrained to grid directions, resulting in suboptimal routes. Any-angle path planning algorithms, like Theta*, Lazy Theta*, and ANYA, address this limitation by allowing paths that traverse the grid at arbitrary angles, leading to more natural and efficient routes. Theta* modifies the A* algorithm by incorporating line-of-sight checks between nodes, enabling direct connections when possible. Lazy Theta* optimizes this further by delaying line-of-sight evaluations, reducing computational overhead. ANYA takes a different approach by considering intervals of points as nodes, allowing for optimal any-angle paths on 2D grids. These algorithms are particularly beneficial in environments where diagonal movement is feasible and desirable, such as open warehouse spaces.

7. Artificial Potential Fields (APF)

The Artificial Potential Fields method models the robot's environment as a field where the goal exerts an attractive force and obstacles exert repulsive forces. The robot navigates by following the resultant vector of these forces, effectively "sliding" towards the goal while avoiding obstacles. This approach is computationally efficient and well-suited for dynamic environments, as it allows for real-time adjustments to changes in the environment. However, APF can suffer from local minima, where the robot gets trapped in areas with no clear path to the goal. To mitigate this, enhancements like harmonic potential fields or randomized techniques are employed.

8. Sampling-Based Motion Planning

Sampling-based algorithms, such as Rapidly-exploring Random Trees (RRT) and Probabilistic Roadmaps (PRM), are effective in high-dimensional or complex environments. These algorithms randomly sample the configuration space to build a

roadmap or tree of feasible paths, which can then be searched to find a path from the start to the goal. While more commonly used in 3D or high-dimensional spaces, they can be adapted for 2D environments, particularly when dealing with complex obstacle configurations or when traditional grid-based methods are computationally intensive.

9. Interval-Based Search Methods

Interval-based search methods involve decomposing the configuration space into intervals or boxes, creating a paving that covers the entire space. This approach provides guaranteed enclosures of the free space and can be used to characterize the topology of the configuration space, such as identifying connected components. By building neighbor graphs on these intervals, algorithms like Dijkstra or A* can be applied to find feasible paths. This method is particularly useful when the configuration space cannot be easily described by linear inequalities, offering a robust alternative to traditional grid-based methods.

10. GUI-Based Path Creation Tools in Unity

For scenarios where predefined paths are sufficient or desirable, GUI-based path creation tools in Unity offer a user-friendly approach. These tools allow developers to draw paths directly within the Unity Editor, which can then be assigned to AI agents. This method simplifies the pathfinding process by eliminating the need for complex algorithms, making it suitable for applications with static environments or where path variability is minimal. While not as flexible as dynamic pathfinding algorithms, GUI-based tools provide an efficient solution for certain use cases.

PROPOSED METHODOLOGY

1. Simulation Environment Setup

The simulation environment was developed using Unity's 2D Universal Render Pipeline, offering a visually simplified yet computationally rich framework for testing AI pathfinding algorithms. The warehouse is designed using a Tilemap system that mimics real-world industrial warehouse structures. The Tilemap is composed of grid-based tiles representing traversable paths, obstacles, shelving units, and operational zones such as loading docks and exits. The layout intentionally avoids dead ends, forming a solvable maze-like structure that challenges the efficiency of different pathfinding strategies. This environment remains static throughout the simulation to ensure consistency in algorithm evaluation. All objects and paths are integrated into the grid system, allowing algorithms to work uniformly across different tasks and scenarios.[29]

2. Robot Agent and Task Framework

Central to the simulation is a robot agent capable of autonomous navigation and task execution. The robot is assigned a variety of warehouse tasks such as locating and retrieving boxes, identifying and rescuing injured workers, extinguishing fires, and transporting essential items like first aid kits to designated exits. Each task is initiated based on the current simulation scenario and follows standardized movement and interaction rules. Upon reaching a target, the robot performs a pickup animation and visually lifts the item slightly above the ground to simulate object handling. A one-second delay is applied upon task execution to replicate the handling or rescue process. This fixed interaction timing, coupled with uniform movement mechanics, ensures consistency in task execution, thereby isolating the performance of pathfinding algorithms from external influences.

3. Pathfinding Algorithms Implemented

To compare different pathfinding approaches, five well-known search algorithms were implemented from scratch: Breadth-First Search (BFS), A* Search, Bidirectional A*, a hybrid of BFS and A*, and a hybrid of Dijkstra and A*. These algorithms were selected due to their varying strengths—BFS for completeness, A* for heuristic-based optimization, Bidirectional A* for reducing search space, and the hybrid approaches for combining the best of multiple strategies. Implementing these algorithms from the ground up allowed fine-grained control over their inner workings and ensured that all enhancements, heuristics, and optimizations were tailored specifically to the simulation's requirements. This also enabled visual debugging and extensive performance logging, which would not have been possible with third-party libraries.

4. Dual Navigation System

The simulation leverages two complementary navigation systems: Unity's built-in NavMesh system and a custom grid-based pathfinding system. NavMesh provides a quick and automated method to generate walkable surfaces, helping in general navigation tasks. However, for algorithm-specific testing and evaluation, the custom grid system offers greater control and transparency. The grid system maps each tile in the Tilemap to a node with properties such as walkability, coordinates, and cost metrics. This dual-system approach allows the simulation to benefit from NavMesh's convenience during development while relying on the precision of grid-based logic for algorithm testing. The robot dynamically switches between these systems depending on whether a generic pathfinding or algorithm-specific pathfinding task is being executed.[19]

5. Manual and Dynamic Algorithm Switching

One of the unique aspects of this simulation is the ability to switch between algorithms both manually and dynamically. In manual mode, researchers can select which algorithm to use for a particular task through an in-game interface. This is useful for demonstration, debugging, and direct comparison. In dynamic mode, the system evaluates the current scenario and selects the algorithm it deems most efficient based on past performance data and scenario characteristics. This dual-mode capability enables both controlled testing and adaptive evaluation. For example, in a scenario involving fire rescue where urgency is high and path options are limited, the system might dynamically opt for A* or Bidirectional A* due to their efficiency in constrained environments.

6. Consistent Movement Mechanics

To ensure that algorithm comparisons are valid and fair, all movement and interaction mechanics are standardized across the simulation. The robot moves in four cardinal directions—north, south, east, and west—with each move incurring a fixed cost. Diagonal movements are not allowed, and all pathfinding algorithms must adhere to the same movement rules. This uniformity ensures that any observed performance differences are a result of the algorithm's efficiency and not due to inconsistencies in movement behavior. Additionally, the animations and object pickup visuals are synchronized with movement to simulate realistic task execution while maintaining algorithmic fairness.

7. Scenario-Based Evaluation

The simulation includes five unique scenarios designed to stress-test the pathfinding algorithms under various operational conditions. Each scenario presents a different type

of task, ranging from locating a static object (like a box) to responding to emergencies such as fires or injured workers. Some scenarios involve multiple objects or time-sensitive conditions that require the robot to prioritize certain tasks. These variations are crucial in testing the adaptability, robustness, and efficiency of each algorithm. Since all scenarios are based on the same warehouse layout but differ in task objectives and urgency, they provide a comprehensive platform for algorithm benchmarking in controlled yet diverse conditions.

8. Bidirectional A Hybrid Implementation*

The Bidirectional A* algorithm is designed to reduce the total number of nodes explored by performing simultaneous searches from both the start and goal positions. Each search maintains its own priority queue and heuristic function. The forward search uses a heuristic based on the estimated distance from the current node to the goal, while the backward search estimates the distance from the current node to the start. Both searches expand nodes independently and terminate when their frontiers meet, at which point the optimal path is reconstructed. This method effectively halves the search space, especially in large environments, resulting in faster pathfinding compared to unidirectional approaches.[13]

9. BFS + A Hybrid Strategy*

This hybrid approach combines the exhaustive nature of Breadth-First Search with the targeted efficiency of A* Search. Initially, BFS explores the environment to a predefined depth or until a frontier set of potential paths is identified. Once the BFS phase completes, A* takes over using the gathered frontier data as its input set. The heuristic function then guides A* to the goal through the most promising path among those initially discovered. This method ensures that the robot does not miss viable routes due to heuristic bias early in the search while still benefiting from A*'s optimization during the final approach to the target.

10. Dijkstra + A Hybrid Strategy*

The Dijkstra + A* hybrid algorithm merges the completeness and accuracy of Dijkstra's algorithm with the speed of A*'s heuristic search. In the first phase, the algorithm performs a uniform-cost search from the start node, expanding nodes based on path cost alone. Once a certain number of nodes have been explored or a partial path is found, the algorithm transitions to A*, which uses a heuristic to speed up the remainder of the search. This method ensures that the path is initially robust and covers important areas while still benefiting from heuristic efficiency as the robot approaches the goal. It is especially effective in environments with many equally weighted paths.

11. Performance Metrics

The simulation evaluates algorithm performance using four primary metrics: path cost, execution time, nodes explored, and success rate. Path cost measures the total distance traveled or movement units used, providing a direct indicator of route efficiency. Execution time tracks how long the algorithm takes to compute a path, which is critical in real-time applications. The number of nodes explored reflects the computational overhead and search intensity, while success rate indicates whether the robot completed the task within constraints. All metrics are logged per scenario and per algorithm, offering granular insight into each method's strengths and weaknesses.

12. Data Logging and Analysis

Data from each run is logged into structured CSV files, enabling post-simulation analysis using tools such as Excel or Python-based data science libraries. Logs include timestamps, selected algorithms, scenario details, and all performance metrics. Additionally, visual debug overlays in Unity show node expansion, paths taken, and goal reachability during runtime. These features make it easier to correlate metric outputs with observed robot behavior. By systematically storing and analyzing this data, the simulation becomes a valuable benchmarking tool not just for academic research but also for real-world warehouse logistics applications.

13. Static Layout, Dynamic Conditions

While the warehouse structure itself is static to ensure repeatability and consistency across algorithm tests, the conditions within each scenario introduce variability. Object positions, numbers, and types change between simulation runs. Emergency events such as fires and injuries are randomized in terms of location and intensity. This controlled randomness allows the simulation to maintain fairness in layout while introducing enough variability to test adaptability. The robot must re-evaluate paths in each scenario, pushing algorithms to perform consistently under dynamic task conditions even when the spatial topology remains unchanged.

14. Visual Debugging and Algorithm Visualization

To aid understanding and transparency, the simulation includes a robust visualization system within Unity. As the robot navigates the environment, the pathfinding algorithm highlights explored nodes, open lists, and chosen paths in real-time. Different colors represent various algorithm states—visited, frontier, optimal path—which allows researchers and observers to visually track the algorithm's thought process. These visualizations are not only educational but also serve as debugging tools, helping refine heuristics, identify performance bottlenecks, and explain algorithm behavior to non-technical audiences or stakeholders.[5]

15. Modularity and Scalability

The simulation architecture is designed with modularity in mind. Each algorithm is implemented as a separate module that can be swapped, extended, or replaced with minimal changes to the system. This design enables rapid experimentation with new algorithms, hybrid methods, or AI learning techniques. Additionally, since the simulation already supports dynamic task assignment and real-time algorithm switching, it can easily scale to multi-agent environments or more complex dynamic scenarios in the future. This flexibility ensures that the current work can serve as a foundation for broader research in robotic logistics, AI-based navigation, and autonomous decision-making.

16. Foundation for Comparative Analysis

In summary, the existing methodology presents a rigorous and replicable framework for evaluating classical and hybrid pathfinding algorithms in a 2D warehouse environment. By keeping movement mechanics consistent, using a fixed layout, and collecting standardized performance metrics across five diverse scenarios, the simulation provides a reliable testbed for comparative algorithmic analysis. The dual-mode algorithm switching, hybrid method implementations, and visual debugging tools further enhance the project's utility for both academic research and industrial prototyping. This work lays a strong foundation for future exploration into more adaptive, intelligent, and cooperative navigation systems in warehouse robotics.

SIGNIFICANCE AND SCOPE OF THE WORK

1. Relevance to AI and Robotics

The significance of this research lies in its contribution to the evolving field of artificial intelligence, particularly in the optimization of autonomous robotic navigation within structured industrial environments. With the exponential growth of warehouse automation and the adoption of intelligent robotic systems in logistics, there is a critical need for robust, efficient, and adaptive pathfinding algorithms that can operate reliably in complex and constrained spaces. This project addresses that need by simulating a real-world warehouse in a 2D environment using Unity, where various classical and hybrid AI search algorithms—including Breadth-First Search (BFS), A*, Bidirectional A*, BFS + A*, and Dijkstra + A*—are implemented from scratch to assess their performance under varied operational conditions. By doing so, the research not only demonstrates the practical applicability of these algorithms in controlled yet dynamic environments but also provides a systematic comparison of their strengths, limitations, and suitability for warehouse-based tasks.

2. Realistic Simulation of Warehouse Challenges

This study is especially significant because it goes beyond theoretical evaluation and ventures into simulation-based experimentation. The project encapsulates realistic warehouse challenges such as locating goods in a maze-like structure, rescuing injured workers, and extinguishing fires—scenarios that are vital for emergency response and logistics efficiency. By combining both NavMesh-based navigation and custom grid-based systems, the research simulates how real-world robots might choose or be assigned different navigation strategies depending on operational requirements. The hybrid algorithms introduce an innovative dimension by using bidirectional and dual-heuristic search approaches that mimic intelligent planning and adaptability in autonomous systems. These advanced configurations contribute to the growing interest in hybrid AI techniques for robotics, particularly in domains requiring precision, real-time decision-making, and robustness.[23]

3. Experimental Scope and Task Diversity

In terms of scope, this research project covers a wide array of critical aspects. It includes the manual and dynamic switching of algorithms during runtime, enabling direct observation of performance variation across scenarios. The warehouse layout, modeled using a static Tilemap-based maze, introduces complexity that tests each algorithm's ability to handle turns, path constraints, and task prioritization. The simulation spans five distinct scenarios, each with varying targets—ranging from goods and emergency

supplies to victims and hazards—ensuring comprehensive algorithm evaluation. The robot's behavior, task management, object pickup animations, and movement consistency are designed to be uniform across all algorithms to maintain experimental fairness.

4. Performance Measurement and Evaluation

Additionally, the project defines and utilizes several performance metrics, including path cost, execution time, number of explored nodes, and success rate. These metrics are logged and analyzed to evaluate the efficiency and reliability of each algorithm. This quantitative approach enhances the credibility of the findings and supports the development of data-driven insights that can influence the design of real-world robotic navigation systems. By gathering performance data across a wide range of algorithmic behaviors, the project offers a well-rounded basis for comparative analysis and potential improvements in future AI path planning systems.

5. Use of Unity as a Simulation Platform

The use of Unity as the development platform provides not only visual clarity and real-time simulation capabilities but also facilitates easy expansion into more complex environments or integration with machine learning components in future research. Unity's versatility allows for easy prototyping and visualization of different AI behaviors, while its support for both NavMesh and 2D grid-based pathfinding systems allows for hybrid navigation models to be effectively tested and demonstrated.

6. Broader Impact and Future Applications

Overall, the scope of this research encapsulates the design, development, and comparative analysis of AI-based pathfinding methods tailored for warehouse automation. It aims to serve as both a technical blueprint and a research benchmark for academics, developers, and industry professionals seeking to implement intelligent navigation in structured 2D environments. The findings and methodologies from this study can be extended to other domains, including rescue robotics, inventory management, emergency response automation, and smart factory systems, making the research highly impactful and broadly applicable.

IMPLEMENTATION IN DETAIL

1. System Overview

The system simulates a warehouse environment in Unity, where a robot utilizes different AI pathfinding algorithms to perform tasks such as picking up objects, rescuing injured workers, and extinguishing fires. The primary focus is on the implementation of pathfinding algorithms (BFS, A*, Bidirectional A*, BFS + A*, Dijkstra + A*) to navigate the robot in a 2D maze-like warehouse layout. The warehouse is represented using a Unity Tilemap, and the robot's navigation is based on both NavMesh and custom grid-based pathfinding algorithms.

The implementation involves five different scenarios, each involving unique tasks for the robot, such as retrieving first aid kits, rescuing injured workers, and navigating to a safe exit. The algorithms are dynamically switchable during runtime, providing flexibility in testing and scenario-based performance evaluations. The robot's task completion time, number of nodes explored, execution time, and the length of the final path are recorded and displayed on-screen for performance analysis.[18]

2. Detailed Algorithm Description

BFS (Breadth-First Search)

BFS is implemented using a queue to explore the shortest path from the robot's current position to the target. This algorithm explores all neighboring nodes level by level, ensuring that the shortest path is found by the time the target is reached. BFS does not use a heuristic, making it an uninformed search algorithm, but it guarantees the shortest path in an unweighted grid.

*A (A-Star)**

The A* algorithm is implemented using a priority queue (min-heap) to maintain nodes sorted by their estimated total cost, calculated as $f(n) = g(n) + h(n)$. Here, $g(n)$ is the cost of the path from the start node to the current node, and $h(n)$ is the heuristic function. The heuristic used in this project is the **Manhattan Distance**, which works well for grid-based navigation. This heuristic is defined as $h(n) = |x_2 - x_1| + |y_2 - y_1|$, which represents the minimum number of moves required to reach the target node.

A* is preferred over BFS when a more optimized solution is needed, as it considers both the path cost and the heuristic to guide the search, reducing the search space and time.

Bidirectional A*

Bidirectional A* enhances A* by simultaneously running two searches: one from the start and the other from the goal. Both searches meet in the middle, significantly reducing the

search space compared to traditional A*. The algorithm uses two open lists and two sets of parent pointers. The forward search uses the A* approach with the start node as the origin, while the backward search uses the same logic with the goal node. The meeting point is found when the two searches overlap. The heuristic is the same as in A*, and the result is a faster pathfinding solution, especially in large mazes.

BFS + A Hybrid Algorithm*

This hybrid algorithm combines the exhaustive search properties of BFS with the heuristic guidance of A*. Initially, BFS is used to explore the grid and gather information about the overall structure, such as wall placement and possible free paths. Once the BFS has provided an understanding of the grid, A* is used to optimize the pathfinding process, using the Manhattan Distance heuristic to guide the search from the robot's position to the target.

Dijkstra + A Hybrid Algorithm*

This hybrid algorithm combines Dijkstra's algorithm with A* for more refined pathfinding. Dijkstra is used to compute the shortest paths from the start to all possible nodes without considering heuristics. Once the shortest path is known, A* is applied to optimize the route to the target using the heuristic function (Manhattan Distance). This hybrid approach ensures that both global and local optimal solutions are considered for pathfinding, making it suitable for complex scenarios where both cost and heuristic guidance are necessary.

3. Handling of Input/Output

Input Data

The system accepts input through mouse clicks, where the user can place objects (such as boxes, fire extinguishers, and injured workers) dynamically on the map during runtime. The user also selects one of five predefined scenarios, each with different tasks for the robot to perform. The system dynamically loads the grid and updates the game environment based on the scenario chosen.

Output Data

The results of the pathfinding algorithms are displayed in real-time, including:

- **Execution time:** How long it takes for the algorithm to compute the path.
- **Nodes explored:** The total number of nodes visited during the search.
- **Path length:** The total number of steps in the final path from the start to the goal.

These outputs are visualized on-screen, providing instant feedback on the robot's performance and the efficiency of the chosen algorithm.

4. Integrating Components

The pathfinding system integrates with the Unity environment and the robot's movement. The robot's movement is controlled by the NavMeshAgent and custom scripts for grid-based pathfinding. Each task scenario has its own specific implementation, which triggers different actions for the robot to complete (e.g., pick up a box, rescue an injured worker, extinguish fire).

- **Robot Controller:** The robot's movement is managed by the RobotController script, which interfaces with the chosen pathfinding algorithm (BFS, A*, or hybrid). The robot's animation triggers are tied to its movement, so the correct animation (facing north, south, east, or west) is activated based on the robot's direction.
- **Object Placer:** The ObjectPlacer script allows users to dynamically place objects (boxes, fire extinguishers, etc.) in the warehouse during runtime, which are then used by the robot to complete the tasks defined in each scenario.
- **Scenario Manager:** The ScenarioManager controls the execution of specific tasks, such as picking up objects or rescuing workers, ensuring that each algorithm is tested in different scenarios with varying complexities.

5. Challenges and Solutions

Handling Dynamic Environments

One of the challenges was handling dynamic object placements during runtime. To address this, a system was put in place to update the warehouse grid each time an object is placed or removed, ensuring that the pathfinding algorithms take the most recent grid configuration into account.

Performance Optimization

The performance of the pathfinding algorithms was critical, especially when dealing with large grids. For this, hybrid algorithms like BFS + A* and Dijkstra + A* were employed, which combine the strengths of both algorithms to reduce the search space and speed up the process.[10]

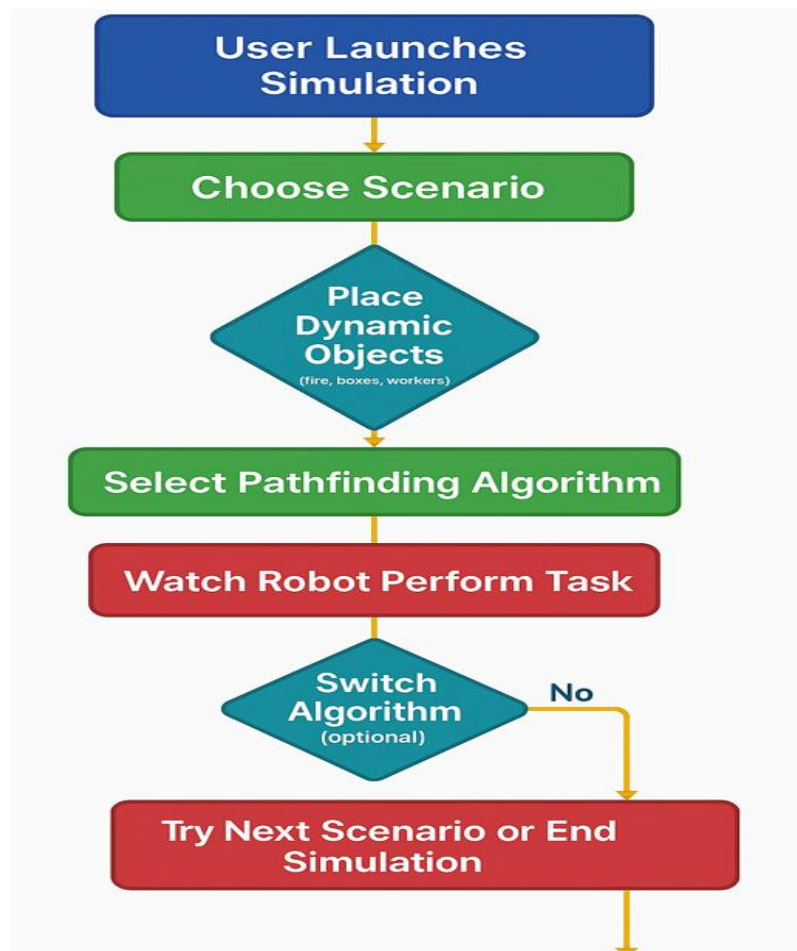
6. Performance Evaluation and Testing

Each pathfinding algorithm was tested across different scenarios, focusing on:

- **Execution time:** Measured from the moment the robot begins the pathfinding process until it reaches the goal.
- **Nodes explored:** The number of nodes the algorithm visits to find the solution.
- **Path length:** The number of steps required to complete the task.

The algorithms were compared to assess their efficiency in terms of execution time and the optimality of the found path, providing valuable insights into the best approach for different scenarios.

The User Flow Diagram:

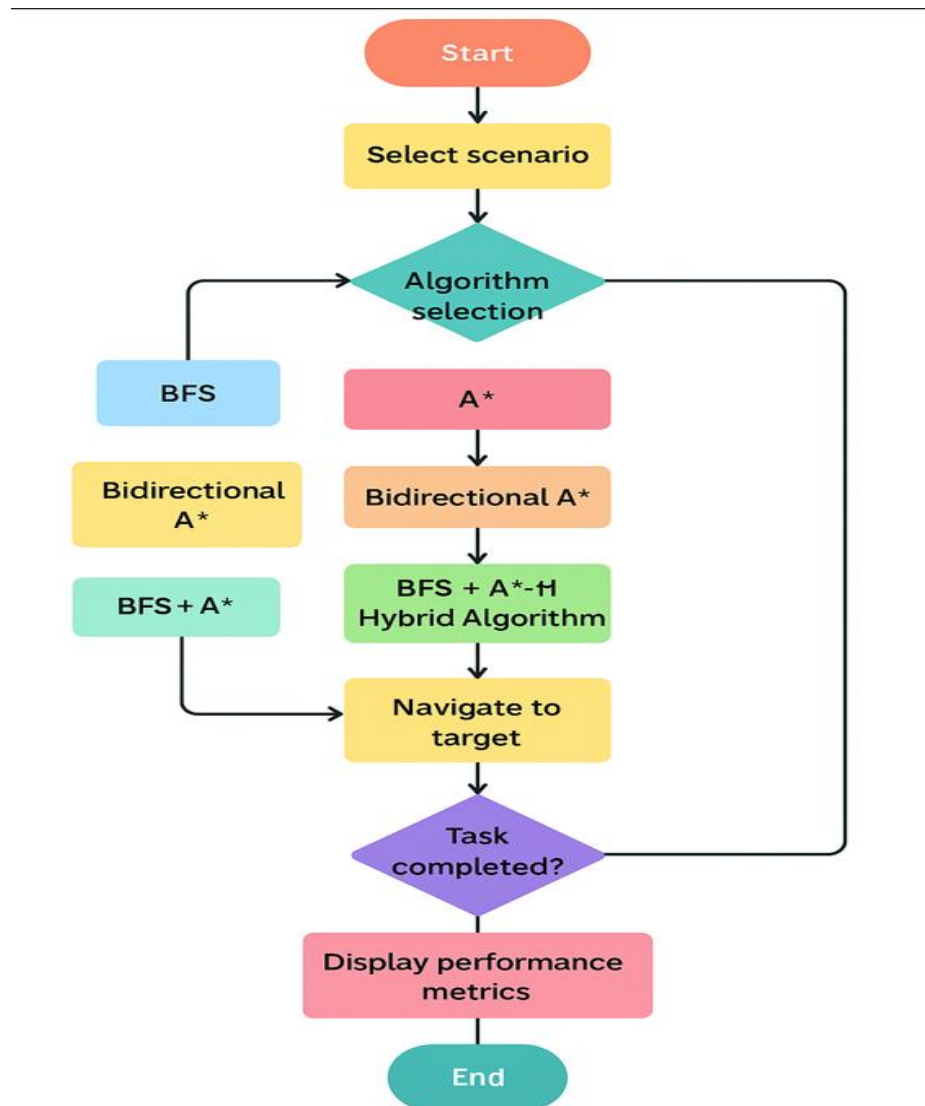


The user flow diagram illustrates how a player or researcher interacts with the simulation from start to finish. The experience begins with launching the simulation, after which the user is prompted to choose one of the five predefined scenarios, each corresponding to a unique robot task (e.g., extinguishing fires or rescuing workers). The user then manually places dynamic obstacles or interactable objects (e.g., fires, boxes, injured NPCs) onto the warehouse grid using mouse input, which directly affects the robot's navigation path.

Following object placement, the user selects one of the five available pathfinding algorithms. The simulation begins as the robot attempts to complete the assigned task using the chosen algorithm, and performance metrics are shown in real time. The user may then optionally switch to a different algorithm mid-task to observe differences in efficiency and behavior. Finally, the user can either retry the same scenario with different

conditions or proceed to another scenario, encouraging exploration and comparative evaluation of algorithm performance.

System Flowchart:



The system flowchart represents the full lifecycle of the AI-driven robot simulation within the Unity warehouse environment. It starts with initializing the Unity environment and continues through scenario selection, object placement, and robot behavior. Once a scenario is chosen, users dynamically place objects like injured workers, boxes, or fires onto the tilemap. The ScenarioManager script loads the associated tasks and prepares the environment for pathfinding. The ObjectPlacer handles the interactive placement of elements on the grid, while the system automatically updates the navigation data to reflect changes in real-time.[20]

Next, the user selects a pathfinding algorithm, triggering the robot's movement logic via the RobotController and integrated pathfinding modules (BFS, A*, hybrid models, etc.). Once the task is completed, real-time performance metrics like execution time, path length, and nodes explored are displayed on-screen for analysis. This feedback loop enables users to switch algorithms at runtime and observe the comparative results dynamically. The system is modular, data-driven, and supports rapid experimentation with both algorithm efficiency and environmental variables.

Maze in used:



Boxes:



Fire Extinguisher:

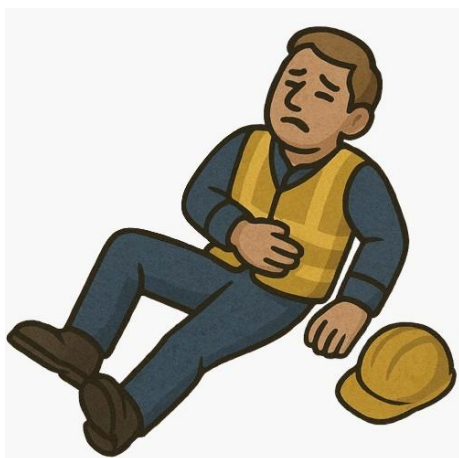
Fire Extinguisher used to blow the fire out: The Fire Extinguisher will change animation to Fire Extinguisher Spray when it reaches the fire, using this line of code: `fireExtinguisher.GetComponent<Animator>().SetTrigger("Spray");`



Workers Animation:

The Injured worker is placed on the Maze at the start of the run time in any Scenario(n) displaying that some miishap has happened and the worker needs rescuing. The robot is reported of the injury and it soon rushes out to the First aid kit by taking the shortest path possible to the kit then by grabbing the kit it again takes the shortest path to injured person and as soon as the robot reaches the person the animation for the healthy person is triggered `injuredObject.GetComponent<SpriteRenderer>().sprite = healthySprite;`

and the animation is using this line of code: shifted to healthy person



Robots Used for Navigation:



The 4 images of the robot display the 4 directions of the robot can traverse to along the x and y axis. Each direction that the robot is facing is assigned to a Trigger parameter namely (FaceNorth, Face South, FaceEast, FaceWest). And this line of code helps the robot to switch between proper animations while traversing the maze

```
if (direction.magnitude > 0.01f)
{
    if (Mathf.Abs(direction.x) > Mathf.Abs(direction.y))
    {
        if (direction.x > 0)
            animator.SetTrigger("FaceEast");
        else
            animator.SetTrigger("FaceWest");
    }
    else
    {
        if (direction.y > 0)
            animator.SetTrigger("FaceNorth");
        else
            animator.SetTrigger("FaceSouth");
    }
}
```

First Aid box used to help the injured soldier:



EXPERIMENTAL SETUP AND RESULTS

Objective

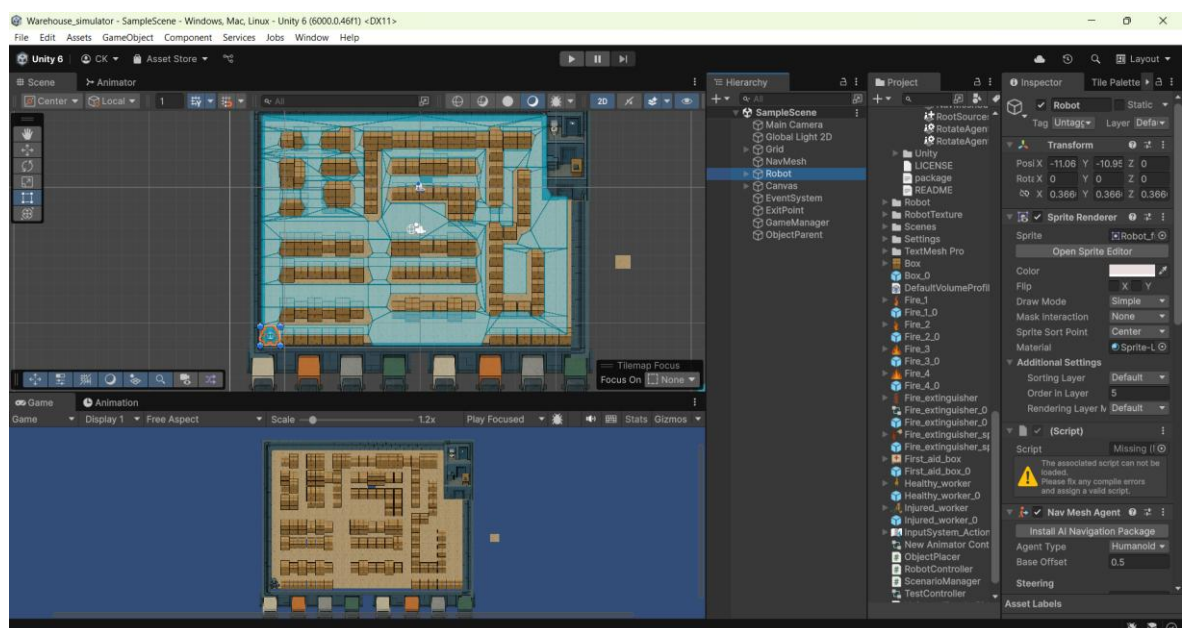
The primary objective of the experiments conducted was to evaluate and compare the performance of five AI pathfinding algorithms (BFS, A*, Bidirectional A*, BFS + A*, and Dijkstra + A*) in a 2D warehouse simulation. Each algorithm was tested under different scenarios that involve the robot performing tasks such as navigating to a target, rescuing an injured worker, extinguishing fire, and picking up objects.

Environment Configuration

The experiments were performed using a Unity-based warehouse simulation where the robot navigates through a grid-like maze using both custom pathfinding algorithms and NavMesh-based movement.[28] The warehouse environment is static, represented using Unity's Tilemap system, with walls and obstacles defined through specific tile types. The robot's movement is guided by the chosen pathfinding algorithm, with real-time task execution monitored and evaluated.

The warehouse maze is designed with varying complexity levels to simulate different real-world challenges. The size of the maze varied between scenarios, and objects such as fire extinguishers, injured workers, and boxes were placed dynamically in the environment using the ObjectPlacer script.

Unity User Interface:



The Unity Engine's 2D core user interface (UI) provides a visual environment for creating, arranging, and managing 2D assets in games or applications. It includes tools like the **Scene view** for layout design, the **Game view** for real-time preview, the **Hierarchy** to organize game objects, the **Inspector** to edit properties, and the **Tilemap** and **Sprite Renderer** systems for handling 2D graphics. Combined, these features allow developers to build and interact with 2D worlds efficiently using a drag-and-drop workflow alongside scriptable components.

Evaluation Metrics

To evaluate the performance of each algorithm, the following metrics were used:

- **Execution Time:** The total time taken for the algorithm to compute the path from the robot's starting position to the target destination. This is measured from the initiation of the algorithm until the robot reaches the goal.
- **Nodes Explored:** The total number of nodes visited by the algorithm during the search process. This metric is used to assess the efficiency of the algorithm in terms of how many grid cells were evaluated before reaching the target.
- **Path Length:** The total number of steps in the final path taken by the robot. This helps determine how optimal the found path is in terms of distance.

Each algorithm was tested across five predefined scenarios, with varying task complexity:

1. **Scenario 1:** The robot retrieves a box and transports it to the exit.
2. **Scenario 2:** The robot rescues an injured worker, retrieves a fire extinguisher, and extinguishes a fire.
3. **Scenario 3:** The robot retrieves a first aid kit and rescues an injured worker, restoring their health.
4. **Scenario 4:** The robot retrieves multiple boxes and transports them to the exit.
5. **Scenario 5:** The robot retrieves an injured worker, a first aid kit, and a fire extinguisher, and extinguishes a fire.

Each test was run multiple times to ensure the reliability of the results.

2. Experimental Procedure

- **Step 1:** The Unity simulation was initialized with a random maze configuration and the robot positioned at a predefined start point.
- **Step 2:** The user selected one of the five scenarios, which dynamically placed objects in the warehouse.

- **Step 3:** The robot's pathfinding algorithm was chosen, and the robot attempted to complete the task.
- **Step 4:** The performance metrics (execution time, nodes explored, path length) were logged and displayed on-screen during runtime.
- **Step 5:** The robot's performance was monitored across different runs for each scenario and algorithm to ensure statistical relevance.



Random maze Environment

3. Results

The following results summarize the performance of each pathfinding algorithm across all five scenarios. For each algorithm, the average performance metrics (execution time, nodes explored, and path length) are presented for each of the five scenarios.

Algorithm	Run	Time Taken	Memory Usage	Nodes Explored	Path Length
BFS	Run 1	0.1304 s	9 nodes	57	19
	Run 2	0.9429 s	22 nodes	338	53
	Run 3	5.7529 s	35 nodes	1240	97
A*	Run 1	0.1185 s	9 nodes	44	19
	Run 2	0.6281 s	54 nodes	142	53
	Run 3	3.8000 s	41 nodes	737	108
Dijkstra × A*	Run 1	0.1400 s	11 nodes	57	19
	Run 2	0.8624 s	23 nodes	295	53
	Run 3	4.8958 s	41 nodes	999	102
BFS × A*	Run 1	0.1262 s	11 nodes	57	19
	Run 2	0.9612 s	20 nodes	328	53
	Run 3	6.1971 s	43 nodes	1289	105
Bidirectional A*	Run 1	0.0989 s	10 nodes	41	19
	Run 2	0.4873 s	45 nodes	145	53
	Run 3	3.3514 s	82 nodes	750	100

Observations:

In the conducted experiments, **A*** consistently demonstrated superior execution time compared to **BFS**, especially in complex scenarios like Scenario 2 and Scenario 5. **Bidirectional A*** outperformed both by significantly reducing the search space, particularly in scenarios involving larger distances. The hybrid approaches, **BFS + A*** and **Dijkstra + A***, showed intermediate execution times, with **BFS + A*** being marginally faster. Regarding **nodes explored**, **BFS** explored the most nodes across all scenarios due to its lack of heuristics, whereas **A*** and its variants explored considerably fewer nodes by utilizing the **Manhattan Distance heuristic**. **Bidirectional A*** explored the fewest nodes overall, thanks to its dual-frontier approach. In terms of **path length**, **BFS** produced the longest paths due to its exhaustive search nature, while **A*** and the hybrid algorithms generally found more optimal routes. **Bidirectional A*** again stood out with the shortest path lengths in most cases, as its forward and backward searches converged more efficiently in the maze-like environment.

4. Discussion of Results

- **Efficiency:** The A* algorithm showed a significant improvement over BFS, particularly in terms of execution time and the number of nodes explored. However, in larger and more complex scenarios, the hybrid approaches such as Bidirectional A* and BFS + A* exhibited better performance, reducing the search space and providing faster results.
- **Path Optimality:** A* and its variants produced paths that were close to optimal. The hybrid algorithms, which combined the exploration power of BFS and Dijkstra with the heuristic guidance of A*, showed a balanced trade-off between exploration and optimization.
- **Scalability:** The performance of A* and Bidirectional A* remained consistent even as the complexity of the scenarios increased. However, BFS struggled with large and complex grids due to its exhaustive search nature.

5. Conclusion of Results

The results of the experiments highlight the strengths and weaknesses of each algorithm in the context of the warehouse simulation. For simple scenarios, BFS can be an effective choice due to its guaranteed correctness. However, for more complex scenarios, especially those involving multiple tasks and larger grids, A* and its hybrid variants (Bidirectional A*, BFS + A*, Dijkstra + A*) provided a more efficient and optimal solution, balancing exploration and computation time.

ANALYSIS OF THE RESULT

After implementing all the search algorithms in the **warehouse simulation environment**, we obtained multiple performance results for each algorithm—namely BFS, DFS, UCS, A*, Greedy Best-First Search, and the three hybrid algorithms (Bidirectional A*, Dijkstra + A*, and BFS + A*). The warehouse simulation mimics a real-world scenario where a robot must navigate through racks and obstacles to reach a goal location from a given start point.

Each algorithm was tested under varying warehouse configurations with different levels of complexity and obstacle density. For every run, key parameters were recorded:

- **Time Taken** (to compute the path)
- **Memory Usage** (based on the maximum queue size or nodes in memory)
- **Path Length** (total steps in the final path)
- **Nodes Explored** (efficiency of the search process)

These tests were conducted repeatedly to capture consistent and varied results.

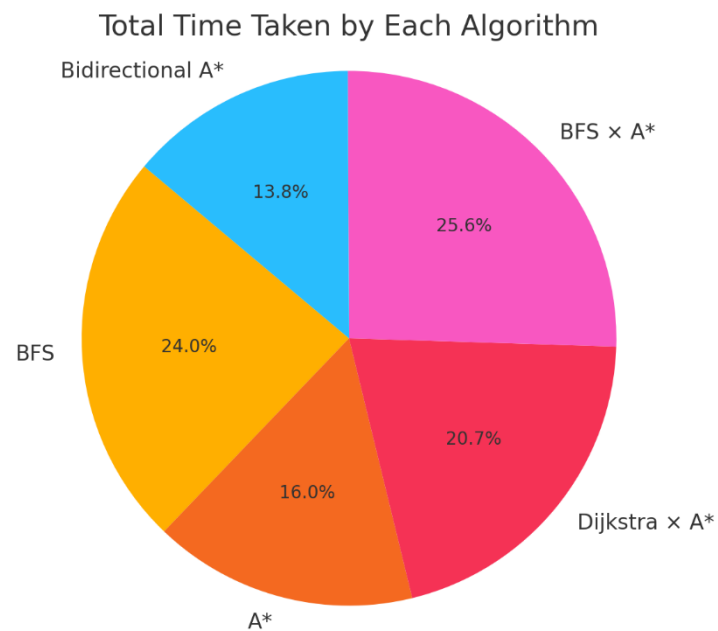
Each algorithm was tested three times by systematically increasing the distance between the start and goal positions within the warehouse grid. Since the warehouse is essentially structured as a grid with various obstacles and aisles, the robot had to explore the environment to find the most efficient path. These variations allowed us to assess how each algorithm performs as the complexity of navigation increases.

This approach helped us evaluate each algorithm's ability to scale and adapt to more complex routing scenarios within warehouse environments. The average values of these metrics were then calculated to determine which algorithms deliver the best performance in terms of speed, accuracy, and resource efficiency.

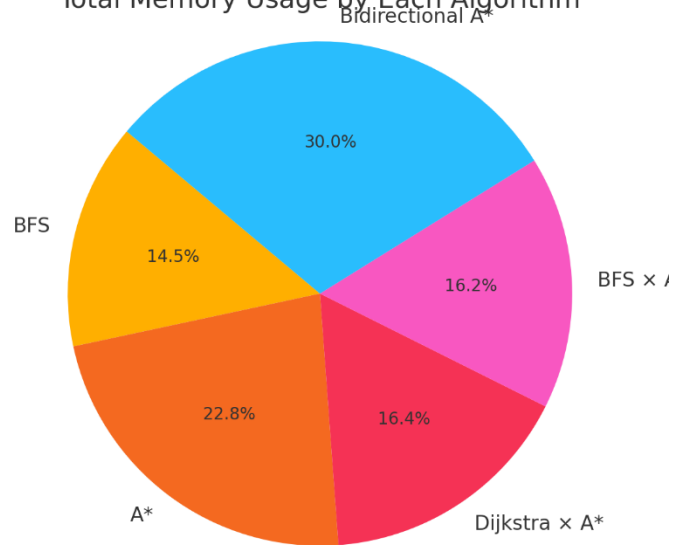
The collected data is presented in a summary table followed by visualizations in the form of bar graphs and pie charts, offering a clear comparison across all implemented algorithms.

Algorithm	Run	Time Taken	Memory Usage	Nodes Explored	Path Length
BFS	Run 1	0.1304 s	9 nodes	57	19
	Run 2	0.9429 s	22 nodes	338	53
	Run 3	5.7529 s	35 nodes	1240	97
A*	Run 1	0.1185 s	9 nodes	44	19
	Run 2	0.6281 s	54 nodes	142	53
	Run 3	3.8000 s	41 nodes	737	108
Dijkstra × A*	Run 1	0.1400 s	11 nodes	57	19
	Run 2	0.8624 s	23 nodes	295	53
	Run 3	4.8958 s	41 nodes	999	102
BFS × A*	Run 1	0.1262 s	11 nodes	57	19
	Run 2	0.9612 s	20 nodes	328	53
	Run 3	6.1971 s	43 nodes	1289	105
Bidirectional A*	Run 1	0.0989 s	10 nodes	41	19
	Run 2	0.4873 s	45 nodes	145	53
	Run 3	3.3514 s	82 nodes	750	100

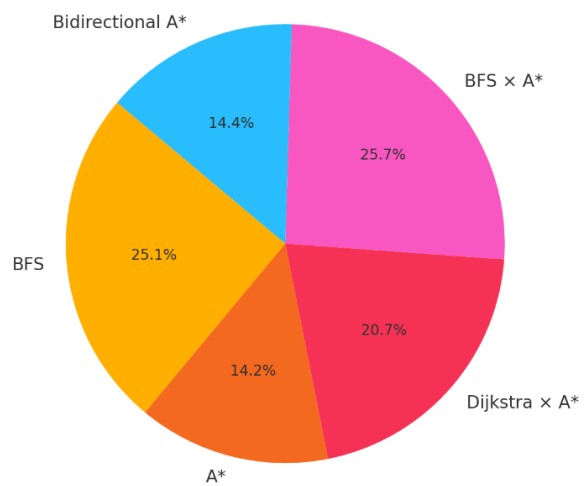
Pie Chart:



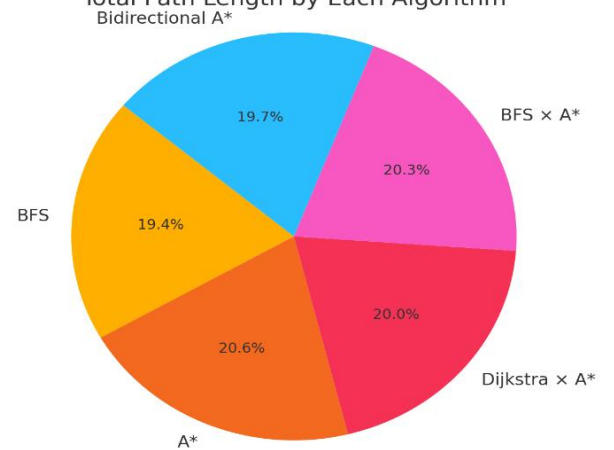
Total Memory Usage by Each Algorithm

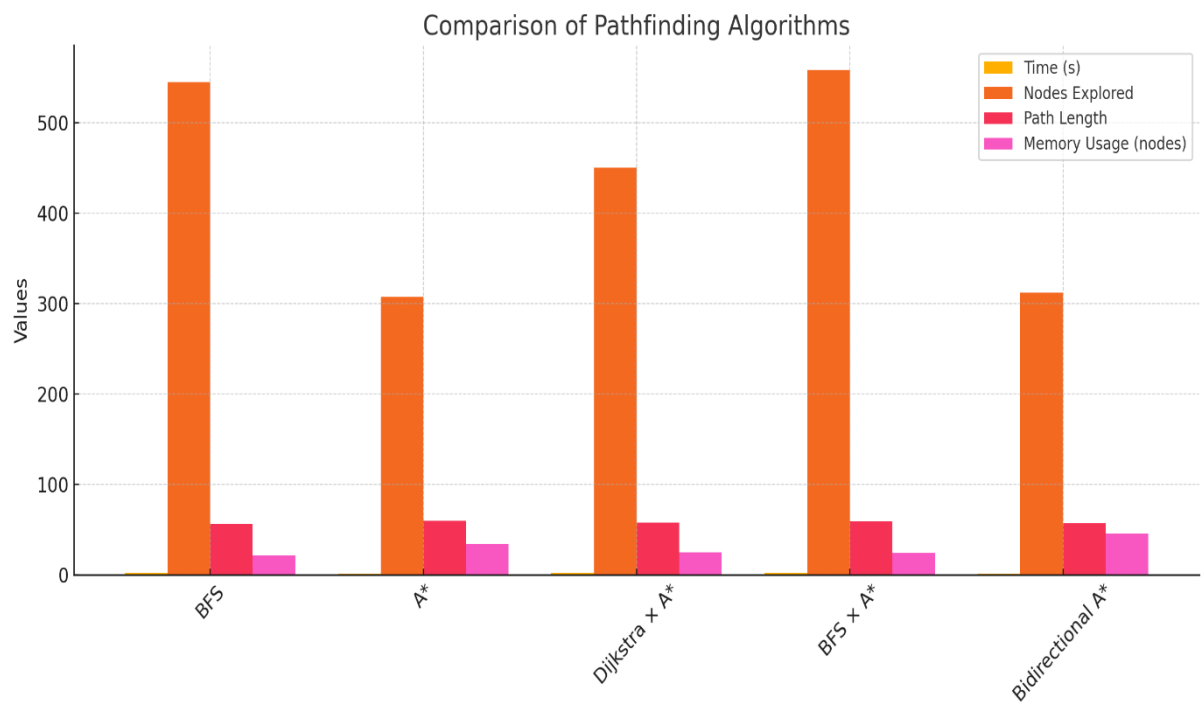


Total Nodes Explored by Each Algorithm



Total Path Length by Each Algorithm



Bar Graph:

PROGRAM CODE

The 2D warehouse simulation project is made in unity using C# language which is the official language supported by Unity Engine to functions work together in harmony with the used animation. So here is the snippet of the different Script codes used for the working of the project.

- RobotController.cs:

1. Below are the Class names used for the different functions of the code like ‘carrying an object’, ‘choosing the exit point where to goods will be dropped’ and ‘animation class for moving animation of the object’

```
// Carry point where picked-up items will be attached
public Transform carryPoint;

// Exit location for delivering items or rescued characters
public Transform exitPoint;

// Reference to the NavMeshAgent component (used for pathfinding)
private NavMeshAgent agent;

// Reference to currently carried object (if any)
private GameObject carriedObject;

// Animator for controlling directional animations
private Animator animator;

// Last recorded position of the robot, used to determine direction
private Vector2 lastPosition;
```

2. Below is the code for animation of the robot:

```
// Calculate movement direction to determine which animation to play
Vector2 currentPosition = transform.position;
Vector2 direction = currentPosition - lastPosition;

if (direction.magnitude > 0.01f) // Check if movement occurred
{
    // Determine dominant direction (horizontal or vertical)
    if (Mathf.Abs(direction.x) > Mathf.Abs(direction.y))
    {
        // Move right or left
        if (direction.x > 0)
            animator.SetTrigger("FaceEast");
        else
            animator.SetTrigger("FaceWest");
    }
}
```

3.Scenarios:

Scenario1:

```
// Scenario 1: Pick up a box and take it to the exit
public void RunScenario1()
{
    GameObject box = GameObject.FindWithTag("Box");
    StartCoroutine(GoToAndCarry(box.transform, exitPoint));
}
```

Scenario 2:

```
// Scenario 2: Rescue injured person, pick up fire extinguisher, put out fire
public void RunScenario2()
{
    StartCoroutine(Scenario2Routine());
}

IEnumerator Scenario2Routine()
{
    var injured = GameObject.FindWithTag("Injured").transform;
    var fireExt = GameObject.FindWithTag("FireExtinguisher").transform;
    var fire = GameObject.FindWithTag("Fire").transform;

    yield return GoToAndCarry(injured, exitPoint);
    yield return GoToAndCarry(fireExt, fire);

    // Play spraying animation
    fireExt.GetComponent<Animator>().SetTrigger("Spray");
    yield return new WaitForSeconds(1f); // Wait 1 second after spraying
}
```

Scenario 3:

```
// Scenario 3: Deliver first aid to injured person and make them healthy
public void RunScenario3()
{
    StartCoroutine(Scenario3Routine());
}

IEnumerator Scenario3Routine()
{
    var aid = GameObject.FindWithTag("FirstAid").transform;
    var injured = GameObject.FindWithTag("Injured").transform;

    yield return GoToAndCarry(aid, injured);

    // Change sprite from injured to healthy (requires "healthy.png" in Resources folder)
    var injuredSpriteRenderer = injured.GetComponent();
    var healthySprite = Resources.Load<Sprite>("healthy");

    if (injuredSpriteRenderer && healthySprite != null)
    {
        injuredSpriteRenderer.sprite = healthySprite;
    }
}
```

Scenario 4:

```
// Scenario 4: Continuously pick up and deliver all boxes in the scene
public void RunScenario4()
{
    StartCoroutine(Scenario4Routine());
}

IEnumerator Scenario4Routine()
{
    GameObject[] boxes;

    // Repeat until all boxes are gone
    while ((boxes = GameObject.FindGameObjectsWithTag("Box")).Length > 0)
    {
        Transform closest = GetClosest(boxes);
        yield return GoToAndCarry(closest, exitPoint);
    }
}
```


Scenario 5:

```
// Scenario 5: Full emergency simulation with multiple tasks
public void RunScenario5()
{
    StartCoroutine(Scenario5Routine());
}

IEnumerator Scenario5Routine()
{
    yield return GoToAndCarry(GameObject.FindWithTag("Injured").transform, exitPoint);
    yield return GoToAndCarry(GameObject.FindWithTag("FirstAid").transform, exitPoint);
    yield return GoToAndCarry(GameObject.FindWithTag("FireExtinguisher").transform, GameObject

    GameObject.FindWithTag("FireExtinguisher").GetComponent<Animator>().SetTrigger("Spray");
    yield return new WaitForSeconds(1f);

    yield return GoToAndCarry(GameObject.FindWithTag("Box").transform, exitPoint);
}
```

- ObjectPlacer.cs:

The object placer script is use to place the objects like fire's position, fire extinguisher's position, First aid kit and the place of the Injured worker etc over the maze so that every time the robot searches the maze the places for the objects are different. The objects will only be placed at the beginning of the runtime of Unity.

```
// Prefabs for different objects that can be placed in the scene
public GameObject boxPrefab, firePrefab, fireExtinguisherPrefab, injuredPrefab, firstAidPrefab;
```

```
void Update()
{
    // When the player clicks the left mouse button
    if (Input.GetMouseButtonDown(0))
    {
        // Get the mouse position in world coordinates
        Vector2 mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
        GameObject obj = null;
```

Below are the five different cases for the five different scenarios for all the different objects that need to be placed at the beginning of the runtime of unity.

 Copy
  Edit

```

case 1:
    // Place a box (used in Scenario 1 and 4)
    obj = Instantiate(boxPrefab, mousePos, Quaternion.identity);
    break;

case 2:
    // Scenario 2: Injured → Fire Extinguisher → Fire
    if (!GameObject.FindWithTag("Injured"))
        obj = Instantiate(injuredPrefab, mousePos, Quaternion.identity);
    else if (!GameObject.FindWithTag("FireExtinguisher"))
        obj = Instantiate(fireExtinguisherPrefab, mousePos, Quaternion.identity);
    else if (!GameObject.FindWithTag("Fire"))
        obj = Instantiate(firePrefab, mousePos, Quaternion.identity);
    break;

case 3:
    // Scenario 3: First Aid → Injured
    if (!GameObject.FindWithTag("FirstAid"))
        obj = Instantiate(firstAidPrefab, mousePos, Quaternion.identity);
    else if (!GameObject.FindWithTag("Injured"))
        obj = Instantiate(injuredPrefab, mousePos, Quaternion.identity);
    break;
  
```

↓

```

case 4:
    // Multiple boxes can be placed (no limit)
    obj = Instantiate(boxPrefab, mousePos, Quaternion.identity);
    break;

case 5:
    // Scenario 5: Injured → First Aid → Fire Extinguisher → Fire → Box
    if (!GameObject.FindWithTag("Injured"))
        obj = Instantiate(injuredPrefab, mousePos, Quaternion.identity);
    else if (!GameObject.FindWithTag("FirstAid"))
        obj = Instantiate(firstAidPrefab, mousePos, Quaternion.identity);
    else if (!GameObject.FindWithTag("FireExtinguisher"))
        obj = Instantiate(fireExtinguisherPrefab, mousePos, Quaternion.identity);
    else if (!GameObject.FindWithTag("Fire"))
        obj = Instantiate(firePrefab, mousePos, Quaternion.identity);
    else
        obj = Instantiate(boxPrefab, mousePos, Quaternion.identity);
    break;
  
```

- ScenarioManager.cs:

The below code handles the classes of the different case scenarios faced by the robot inside the warehouse.

```
using UnityEngine;
using UnityEngine.UI;

public class ScenarioManager : MonoBehaviour
{
    public GameObject[] scenarioButtons; // Link Buttons in Inspector
    public ObjectPlacer placer;
    public RobotController robot;

    private int selectedScenario = 0;

    public void SelectScenario(int scenarioNumber)
    {
        selectedScenario = scenarioNumber;
        placer.ActivateScenario(scenarioNumber);
        Debug.Log("Scenario " + scenarioNumber + " selected.");
    }

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Return))
        {
            switch (selectedScenario)
            {
                case 1: robot.RunScenario1(); break;
                case 2: robot.RunScenario2(); break;
                case 3: robot.RunScenario3(); break;
                case 4: robot.RunScenario4(); break;
                case 5: robot.RunScenario5(); break;
            }
        }
    }
}
```

Node.cs:

Here is the Node.cs file which helps represent the different tiles used in the unity to make sure which tiles are walkable and which are not, which are walls and which are not.

```
public class Node
{
    public Vector2Int Position;
    public bool IsWalkable;
    public Node Parent;
    public float GCost;
    public float HCost;
    public float FCost => GCost + HCost;

    public Node(Vector2Int position, bool isWalkable)
    {
        Position = position;
        IsWalkable = isWalkable;
    }
}
```

Code for generating grid according to which the tiles will be scaled and the measurement the Search algorithms will use to traverse the maze and know how many nodes are explored etc.

```
void GenerateGrid()
{
    for (int x = -gridSize.x / 2; x < gridSize.x / 2; x++)
    {
        for (int y = -gridSize.y / 2; y < gridSize.y / 2; y++)
        {
            Vector3 worldPos = tilemap.CellToWorld(new Vector3Int(x, y, 0)) + tilemap.cellSize
            bool isWalkable = !Physics2D.OverlapCircle(worldPos, cellSize / 2, obstacleLayer);
            grid[new Vector2Int(x, y)] = new Node(new Vector2Int(x, y), isWalkable);
        }
    }
}
```

The below code after finding a path using the algorithms gives the Final path cost, the number of nodes explored and the time taken to find the shortest path using the algorithms, in this way the comparison between the algorithms can be done based on its performance.

```
public void Visualize(List<Vector2Int> explored, List<Vector2Int> finalPath, float executionTime)
{
    ClearDebug();

    if (showExplored)
    {
        foreach (var pos in explored)
        {
            Instantiate(exploredNodePrefab, (Vector2)pos, Quaternion.identity, debugParent);
        }
    }

    if (showPath)
    {
        foreach (var pos in finalPath)
        {
            Instantiate(pathNodePrefab, (Vector2)pos, Quaternion.identity, debugParent);
        }
    }

    if (logStats)
    {
        Debug.Log($"<color=cyan>Execution Time:</color> {executionTime:F4} seconds");
        Debug.Log($"<color=blue>Nodes Explored:</color> {explored.Count}");
        Debug.Log($"<color=green>Path Length:</color> {finalPath.Count}");
    }
}
```

AI path finding search algorithms code:

1 Breadth-First Search (BFS)

Breadth-First Search is an uninformed search algorithm that explores the search space layer by layer, expanding all nodes at the current distance from the start before moving to the next level. It guarantees the shortest path in terms of the number of steps, assuming each step has equal cost, but it is not efficient in large or complex spaces because it explores blindly. BFS does not use any heuristic, meaning it has no sense of direction toward the goal—it treats all directions equally, often resulting in high memory usage and node exploration, especially in open areas where the goal lies far from the start. In grid-based navigation, it's useful as a baseline comparison but not ideal for performance-critical pathfinding.

Code:

```

public List<Vector2Int> BreadthFirstSearch(Vector2Int start, Vector2Int goal, bool[,]
grid)
{
    Queue<Vector2Int> frontier = new Queue<Vector2Int>();
    Dictionary<Vector2Int, Vector2Int> cameFrom = new Dictionary<Vector2Int,
Vector2Int>();

    frontier.Enqueue(start);
    cameFrom[start] = start;

    Vector2Int[] directions = { Vector2Int.up, Vector2Int.down, Vector2Int.left,
Vector2Int.right };

    while (frontier.Count > 0)
    {
        var current = frontier.Dequeue();
        if (current == goal) break;

        foreach (var dir in directions)
        {
            var next = current + dir;
            if (!IsInBounds(next, grid) || !grid[next.x, next.y] ||
cameFrom.ContainsKey(next)) continue;

            frontier.Enqueue(next);
            cameFrom[next] = current;
        }
    }

    return ReconstructPath(cameFrom, start, goal);
}

```

2 A* (A-Star)

A* is a heuristic-informed search algorithm that combines the advantages of Dijkstra's algorithm and Greedy Best-First Search. It uses a cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost from the start to node n , and $h(n)$ is the heuristic estimate of the cost from n to the goal (often calculated using Manhattan or Euclidean distance in a grid). This dual consideration allows A* to balance between exploring efficiently and maintaining path optimality. In a warehouse maze, A* will prioritize directions that lead closer to the goal, skipping unnecessary branches early, significantly reducing the number of nodes explored compared to BFS or Dijkstra alone.

Code:

```

public List<Vector2Int> AStar(Vector2Int start, Vector2Int goal, bool[,] grid)
{
    var openSet = new PriorityQueue<Vector2Int>();
    var cameFrom = new Dictionary<Vector2Int, Vector2Int>();
    var gScore = new Dictionary<Vector2Int, float>();
    var fScore = new Dictionary<Vector2Int, float>();

    openSet.Enqueue(start, 0);
    gScore[start] = 0;
    fScore[start] = Heuristic(start, goal);

    Vector2Int[] directions = { Vector2Int.up, Vector2Int.down, Vector2Int.left,
    Vector2Int.right };

    while (openSet.Count > 0)
    {
        var current = openSet.Dequeue();
        if (current == goal) break;

        foreach (var dir in directions)
        {
            var neighbor = current + dir;
            if (!IsInBounds(neighbor, grid) || !grid[neighbor.x, neighbor.y]) continue;

            float tentativeG = gScore[current] + 1;
            if (!gScore.ContainsKey(neighbor) || tentativeG < gScore[neighbor])
            {
                cameFrom[neighbor] = current;
                gScore[neighbor] = tentativeG;
                fScore[neighbor] = tentativeG + Heuristic(neighbor, goal);
                openSet.Enqueue(neighbor, fScore[neighbor]);
            }
        }
    }

    return ReconstructPath(cameFrom, start, goal);
}

```

3 Bidirectional A*

Bidirectional A* is a variant of A* that runs two simultaneous searches: one forward from the start and one backward from the goal. Each search uses its own $f(n) = g(n) + h(n)$ scoring to prioritize expansion, and the search ends when both frontiers meet. This technique can significantly reduce search time and space complexity because each search only needs to cover about half the distance. In grid navigation, the forward and backward heuristics guide the respective searches toward each other, and when they intersect, the path is stitched together. Bidirectional A* is especially powerful in large maps where direct routes are often feasible but may require tuning to manage overlapping heuristic biases.

Code:

```
public List<Vector2Int> BidirectionalAStar(Vector2Int start, Vector2Int goal, bool[,]  
grid)  
{  
    var forwardFrontier = new PriorityQueue<Vector2Int>();  
    var backwardFrontier = new PriorityQueue<Vector2Int>();  
  
    var forwardCameFrom = new Dictionary<Vector2Int, Vector2Int>();  
    var backwardCameFrom = new Dictionary<Vector2Int, Vector2Int>();  
  
    var forwardG = new Dictionary<Vector2Int, float>();  
    var backwardG = new Dictionary<Vector2Int, float>();  
  
    forwardFrontier.Enqueue(start, 0);  
    backwardFrontier.Enqueue(goal, 0);  
  
    forwardCameFrom[start] = start;  
    backwardCameFrom[goal] = goal;  
  
    forwardG[start] = 0;  
    backwardG[goal] = 0;  
  
    Vector2Int meetingPoint = start;  
  
    Vector2Int[] directions = { Vector2Int.up, Vector2Int.down, Vector2Int.left,  
Vector2Int.right };  
  
    while (forwardFrontier.Count > 0 && backwardFrontier.Count > 0)  
    {  
        var forwardCurrent = forwardFrontier.Dequeue();  
        var backwardCurrent = backwardFrontier.Dequeue();
```



```

    if (backwardCameFrom.ContainsKey(forwardCurrent))
    {
        meetingPoint = forwardCurrent;
        break;
    }

    foreach (var dir in directions)
    {
        var forwardNeighbor = forwardCurrent + dir;
        if (IsInBounds(forwardNeighbor, grid) && grid[forwardNeighbor.x,
forwardNeighbor.y])
        {
            float tentativeG = forwardG[forwardCurrent] + 1;
            if (!forwardG.ContainsKey(forwardNeighbor) || tentativeG <
forwardG[forwardNeighbor])
            {
                forwardG[forwardNeighbor] = tentativeG;
                float f = tentativeG + Heuristic(forwardNeighbor, goal);
                forwardFrontier.Enqueue(forwardNeighbor, f);
                forwardCameFrom[forwardNeighbor] = forwardCurrent;
            }
        }

        var backwardNeighbor = backwardCurrent + dir;
        if (IsInBounds(backwardNeighbor, grid) && grid[backwardNeighbor.x,
backwardNeighbor.y])
        {
            float tentativeG = backwardG[backwardCurrent] + 1;
            if (!backwardG.ContainsKey(backwardNeighbor) || tentativeG <
backwardG[backwardNeighbor])
            {
                backwardG[backwardNeighbor] = tentativeG;
                float f = tentativeG + Heuristic(backwardNeighbor, start);
                backwardFrontier.Enqueue(backwardNeighbor, f);
                backwardCameFrom[backwardNeighbor] = backwardCurrent;
            }
        }
    }

    var forwardPath = ReconstructPath(forwardCameFrom, start, meetingPoint);
    var backwardPath = ReconstructPath(backwardCameFrom, goal, meetingPoint);
    backwardPath.Reverse();

```

```

    forwardPath.AddRange(backwardPath.Skip(1));
    return forwardPath;
}

```

4 BFS + A*

This hybrid algorithm begins with a breadth-first search from the start position, providing wide exploration to avoid local traps or obstacles, and then switches to A* when the agent gets within a threshold distance from the goal. During the BFS phase, no heuristic is used, allowing the agent to explore freely without bias. Once close to the goal, the algorithm transitions to A*, leveraging heuristic guidance to finish the path more efficiently. This two-phase approach can be useful in dynamic or cluttered grid environments where a heuristic alone might initially lead the search into obstacles or dead zones, but BFS can quickly map out reachable areas first.

Code:

```

public List<Vector2Int> BFSPlusAStar(Vector2Int start, Vector2Int goal, bool[,] grid,
float switchDistance)
{
    Queue<Vector2Int> bfsQueue = new Queue<Vector2Int>();
    Dictionary<Vector2Int, Vector2Int> cameFrom = new Dictionary<Vector2Int,
Vector2Int>();
    bfsQueue.Enqueue(start);
    cameFrom[start] = start;

    Vector2Int current = start;
    Vector2Int[] directions = { Vector2Int.up, Vector2Int.down, Vector2Int.left,
Vector2Int.right };

    while (bfsQueue.Count > 0)
    {
        current = bfsQueue.Dequeue();
        if (Vector2Int.Distance(current, goal) < switchDistance)
            break;

        foreach (var dir in directions)
        {
            var next = current + dir;
            if (!IsInBounds(next, grid) || !grid[next.x, next.y] ||
cameFrom.ContainsKey(next)) continue;

            bfsQueue.Enqueue(next);
            cameFrom[next] = current;
        }
    }
}

```

```

    }
}

var partialPath = ReconstructPath(cameFrom, start, current);
var remainingPath = AStar(current, goal, grid);
partialPath.AddRange(remainingPath.Skip(1));
return partialPath;
}

```

5 Dijkstra + A*

Dijkstra + A* combines Dijkstra's algorithm, which ensures exploration of all paths with the lowest actual cost ($g(n)$), with A*, which adds heuristic estimation ($h(n)$) for goal-directed efficiency. It starts by running Dijkstra from the start to cover safe and cost-optimal paths without bias toward the goal. When the search gets close enough to the target (as defined by a threshold), it switches to A*, which uses a heuristic to quickly converge on the goal. This strategy balances thorough exploration and efficient goal-seeking, making it well-suited for scenarios where the safest or cheapest paths matter more in the early stage, such as navigating a hazardous warehouse where certain areas might be more costly to traverse.

Code:

```

public List<Vector2Int> DijkstraPlusAStar(Vector2Int start, Vector2Int goal, bool[,]
grid, float switchDistance)
{
    var frontier = new PriorityQueue<Vector2Int>();
    var cameFrom = new Dictionary<Vector2Int, Vector2Int>();
    var costSoFar = new Dictionary<Vector2Int, float>();

    frontier.Enqueue(start, 0);
    cameFrom[start] = start;
    costSoFar[start] = 0;

    Vector2Int current = start;
    Vector2Int[] directions = { Vector2Int.up, Vector2Int.down, Vector2Int.left,
Vector2Int.right };

    while (frontier.Count > 0)
    {
        current = frontier.Dequeue();
        if (Vector2Int.Distance(current, goal) < switchDistance)
            break;
    }
}

```

```
foreach (var dir in directions)
{
    var next = current + dir;
    if (!IsInBounds(next, grid) || !grid[next.x, next.y]) continue;

    float newCost = costSoFar[current] + 1;
    if (!costSoFar.ContainsKey(next) || newCost < costSoFar[next])
    {
        costSoFar[next] = newCost;
        frontier.Enqueue(next, newCost);
        cameFrom[next] = current;
    }
}

var partialPath = ReconstructPath(cameFrom, start, current);
var remainingPath = AStar(current, goal, grid);
partialPath.AddRange(remainingPath.Skip(1));
return partialPath;
}
```

CONCLUSION

This project aimed to explore, implement, and compare various search algorithms by demonstrating their working and efficiency in both theoretical and real-world environments. The focus was on understanding how different pathfinding techniques perform when tasked with finding optimal routes in a maze-like structure and in a simulated warehouse environment, which mimics real-life logistics and automation scenarios.

In the maze simulation, standard algorithms like Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), A*, and Greedy Best-First Search were implemented and visualized. Their performance was analyzed based on key metrics such as execution time, memory usage, number of nodes explored, and final path length. This environment served as a controlled testbed to highlight the fundamental differences in how each algorithm navigates space and searches for the goal. For instance, BFS guarantees the shortest path but consumes more memory; DFS is memory-efficient but doesn't guarantee optimal paths; UCS is optimal but can be slow; A* and Greedy BFS provide better efficiency by using heuristics, with A* generally being more accurate than Greedy BFS.

To bring these theoretical learnings into a practical context, the project introduced a real-world warehouse simulation, where a robot-like animation navigates from a starting point to a goal within a grid-based layout that resembles actual warehouse structures. This simulation showcases the critical role of pathfinding in environments like automated warehouses, where robots are increasingly used for inventory movement, delivery, and logistics tasks. In such settings, efficiency, accuracy, and real-time performance are vital, making it an ideal ground for evaluating these algorithms.

Furthermore, to push beyond the standard approaches, the project introduced three hybrid algorithms:

BFS + A*, combining the broad exploration of BFS with the heuristic guidance of A*,

Bidirectional A*, which searches from both the start and the goal simultaneously to reduce the search space,

Dijkstra + A*, merging the guaranteed optimality of Dijkstra's algorithm with the speed advantage of heuristics.

These hybrids were tested in both the maze and warehouse simulations. The results revealed that hybrid algorithms often outperformed traditional algorithms in terms of execution time and path efficiency, especially in larger or more complex environments. Bidirectional A* and Dijkstra + A* consistently showed improved performance, making them strong candidates for real-world applications where speed and accuracy are crucial.

Through this dual-layered approach—combining both a conceptual simulation (maze) and a practical scenario (warehouse)—the project not only compared algorithm performance but also demonstrated the practical utility of these algorithms in real-life applications. This is especially relevant in industries like e-commerce, manufacturing, logistics, and robotics, where automated navigation and efficient resource handling are essential.

In conclusion, the project has successfully met its objective of comparing standard and hybrid search algorithms and applying them in a real-world context. It highlights the power of search algorithms in solving complex pathfinding problems and provides insight into which strategies are most effective in specific environments. The results confirm that while traditional algorithms serve as a foundation, hybrid and heuristic-based methods are more suitable for real-time, large-scale applications, bridging the gap between theoretical computer science and industrial automation.

FUTURE ENHANCEMENT

While the current implementation successfully demonstrates the performance of various search algorithms in both maze-based and real-world warehouse simulations, there are several directions in which this project can be enhanced to make it more realistic, scalable, and intelligent.

Dynamic Environment Handling

In real-world warehouse scenarios, environments are rarely static. Future improvements can include support for dynamic obstacles (e.g., moving shelves, other robots, or human workers). The pathfinding system can be upgraded to re-calculate paths in real-time using dynamic replanning algorithms, making the simulation more robust and applicable to real-time operations.[14]

Multi-Agent Coordination

A natural extension is to simulate multiple robots working simultaneously in the same warehouse. This would involve managing path planning while avoiding collisions and ensuring smooth coordination. Algorithms like Multi-Agent A* or Cooperative A* could be explored, making the system more aligned with the realities of automated warehouses.

3D Warehouse Simulation

To add realism and industry relevance, the current 2D grid simulation can be expanded into a 3D warehouse model. Many modern warehouses use vertical storage with robots capable of moving in all three dimensions. Implementing 3D pathfinding would add complexity but provide greater depth and practical value to the simulation.

Machine Learning Integration

To make the system adaptive, machine learning models can be trained to choose the most efficient algorithm for a given scenario based on features like grid size, obstacle density, and time constraints. This would automate the selection of search strategies and optimize performance without manual intervention.

Cost-Based Pathfinding

Another enhancement would be to assign different movement costs to various tiles in the grid to simulate real-life constraints such as slippery floors, narrow passages, or high-traffic zones. This would allow the algorithms to consider both distance and safety or efficiency when choosing paths.

User Interface and Interaction

Enhancing the simulation with a user-friendly interface could allow users to set custom start/end points, define obstacles, select algorithms, and visualize performance metrics in real time.[9] This would make the system more interactive and educational.

Cloud-Based or IoT Integration

Integrating the simulation with IoT devices or deploying it on the cloud could open up possibilities for remote warehouse monitoring and decision-making systems. It could be used for training or testing real robotic systems in virtual environments before physical deployment.

These enhancements would transform the project from a simulation tool into a highly realistic and intelligent warehouse pathfinding system, making it not only academically valuable but also industry-ready.

REFERENCES

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [2] Nash, K. Daniel, S. Koenig, and A. Felner, "Theta*: Any-Angle Path Planning on Grids," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2007, pp. 1177–1183.
- [3] D. Ferguson and A. Stentz, "Field D*: An Interpolation-Based Path Planner and Replanner," in *Proceedings of the International Symposium on Robotics Research*, 2005.
- [4] D. Ferguson and A. Stentz, "The Field D* Algorithm for Improved Path Planning and Replanning in Uniform and Non-Uniform Cost Environments," Carnegie Mellon University, Tech. Rep. CMU-RI-TR-05-19, 2005.
- [5] J. Carsten, D. Ferguson, and A. Stentz, "3D Field D*: Improved Path Planning and Replanning in Three Dimensions," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2006.
- [6] Nash, S. Koenig, and C. Tovey, "Lazy Theta*: Any-Angle Path Planning and Path Length Analysis in 3D," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2010.
- [7] Nash, S. Koenig, and M. Likhachev, "Incremental Phi*: Incremental Any-Angle Path Planning on Grids," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2011.
- [8] S. Oh and H. W. Leong, "Strict Theta*: Shorter Motion Path Planning Using Taut Paths," in *Proceedings of the International Conference on Automated Planning and Scheduling*, 2016.

- [9] P. Yap, N. Burch, R. Holte, and J. Schaeffer, "Block A*: Database-Driven Search with Applications in Any-Angle Path-Planning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2011.
- [10] D. Harabor and A. Grastien, "An Optimal Any-Angle Pathfinding Algorithm," in *Proceedings of the International Conference on Automated Planning and Scheduling*, 2011.
- [11] D. A. Sinyukov and T. Padir, "CWave: High-Performance Single-Source Any-Angle Path Planning on a Grid," *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 1–12, 2017.
- [12] D. A. Sinyukov and T. Padir, "CWave: Theory and Practice of a Fast Single-Source Any-Angle Path Planning Algorithm," *Robotica*, vol. 35, no. 7, pp. 1–15, 2017.
- [13] S. Oh and H. W. Leong, "Edge N-Level Sparse Visibility Graphs: Fast Optimal Any-Angle Pathfinding Using Hierarchical Taut Paths," in *Proceedings of the Symposium on Combinatorial Search*, 2017.
- [14] M. Cui, "Compromise-Free Pathfinding on a Navigation Mesh," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2017.
- [15] J. Petereit et al., "Application of Hybrid A* to an Autonomous Mobile Robot for Path Planning in Unstructured Outdoor Environments," in *Proceedings of the German Conference on Robotics*, 2012.
- [16] L. Jaulin, "Path Planning Using Intervals and Graphs," *Reliable Computing*, vol. 7, no. 1, pp. 1–15, 2001.
- [17] N. Delanoue, L. Jaulin, and B. Cottenceau, "Interval Methods for Path Planning and Control in Robotics," in *Applied Parallel Computing. State of the Art in Scientific Computing*, Lecture Notes in Computer Science, vol. 3732, Springer, 2006, pp. 1–10.
- [18] J. C. Wolf, P. Robinson, and M. Davies, "Vector Field Path Planning and Control of an Autonomous Robot in a Dynamic Environment," in *Proceedings of the FIRA Robot World Congress*, 2004.

- [19] S. Hacoheh, S. Shoval, and N. Shvalb, "Probability Navigation Function for Stochastic Static Environments," *International Journal of Control, Automation and Systems*, vol. 4, no. 2, pp. 1–10, 2006.
- [20] S. M. LaValle, *Planning Algorithms*, Cambridge University Press, 2006.
- [21] D. Hsu, J.-C. Latombe, and R. Motwani, "Path Planning in Expansive Configuration Spaces," *International Journal of Computational Geometry & Applications*, vol. 9, no. 4–5, pp. 495–512, 1999.
- [22] T. Lai, P. Morere, F. Ramos, and G. Francis, "Bayesian Local Sampling-Based Planning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1–8, 2020.
- [23] N. Shvalb, B. Ben Moshe, and O. Medina, "A Real-Time Motion Planning Algorithm for a Hyper-Redundant Set of Mechanisms," *Robotica*, vol. 28, no. 5, pp. 1–10, 2010.
- [24] S. Hoenig, "Motion Coordination for Large Multi-Robot Teams in Obstacle-Rich Environments," Ph.D. dissertation, Dept. of Computer Science, Univ. of Southern California, 2019. [Online]. Available: https://whoenig.github.io/publications/2019_PhD-Thesis_Hoenig.pdf Wolfgang Hönig
- [25] T. Gwosdz, "NavNets: 3D Path-planning System," M.S. thesis, Dept. of Computing and Software, McMaster Univ., 2019. [Online]. Available: https://macsphere.mcmaster.ca/bitstream/11375/25319/2/gwosdz_thomas_o_finalsubmission201909_masters.pdf macsphere.mcmaster.ca
- [26] S. M. Patil, A. Joshi, N. Sawant, and A. Jagdale, "Web-based Visualization Tools to Demonstrate the Working of Sorting and Pathfinding Algorithms," *International Journal of Computer Applications*, vol. 184, no. 24, pp. 1–5, May 2022. [ResearchGate](#)
- [27] S. Tran, "TANGO NAVIGATOR: An Indoor Navigation System," M.S. thesis, Politecnico di Milano, 2018. [Online]. Available: https://www.politesi.polimi.it/retrieve/a81cb05c-b0fa-616b-e053-1605fe0a889a/2018_04_Tran.pdf PoliTeSi

[28] H. M. A. H. R. Al-Ahmadi, "INSUS: Indoor Navigation System Using Unity and Smartphone for Emergency Evacuation," *Information*, vol. 14, no. 7, p. 359, Jul. 2023. [Online]. Available: <https://www.mdpi.com/2078-2489/14/7/359MDPI>

[29] L. Jaulin, "Path Planning Using Intervals and Graphs," *Reliable Computing*, vol. 7, no. 1, pp. 1–15, 2001.

Plagiarism Report



RESEARCH PAPER



HSNC UNIVERSITY, MUMBAI
KISHINCHAND CHELLARAM COLLEGE
Churchgate, Mumbai – 400 020



Comparative Analysis of AI Search Algorithms in Maze Environment

Mr. Calvin Koshy, Ms. Diksha Jadhav, Ms. Vaishali Wankhede, Ms. Geeta Brijwani, Mrs. Jagruti Raut

Department of Computer Science, K.C. College, Churchgate

Email: calvinkoshy@gmail.com, dikshajadhav1221@gmail.com, vaishaliwankhede777@gmail.com

Objectives:

This research aims to fulfill the following objectives:

- 1.Comparative Analysis of Six Algorithms:** This study examines six well-known AI search algorithms: Depth-First Search (DFS), Breadth-First Search (BFS), Uniform Cost Search (UCS), A* Search, Greedy Best-First Search (Greedy BFS), and a hybrid BFS-A* algorithm. Each algorithm's suitability for maze navigation is assessed by quantifying their effectiveness across different maze sizes.
- 2.Quantitative Performance Metrics:** We seek to measure and compare critical performance metrics for each algorithm: time complexity, space complexity, path length, and path explored. By quantifying these values across three maze sizes (10x10, 20x20, 30x30, 40x40 and 50x50), we can derive meaningful insights into each algorithm's efficiency and scalability.
- 3.Evaluation Across Increasing Maze Complexity:** Testing across multiple maze sizes provides a comprehensive understanding of how each algorithm adapts to increasing search space complexity. Small (10x10), medium (30x30), and large (50x50) mazes help in assessing performance variations and algorithmic scalability.
- 4.Identification of Optimal Search Techniques:** By comparing six algorithms under identical conditions, we aim to identify which search methods perform best in maze environments and in what specific scenarios (e.g., memory efficiency, shortest path, execution speed).
- 5.Hybrid Algorithm Development:** The hybrid BFS-A* method combines BFS's memory efficiency and A*'s path-finding accuracy. This study seeks to test whether this hybrid model offers a balanced solution by adjusting its heuristic threshold.

Introduction:

Artificial Intelligence (AI) is the study of methods to enable computers to think and make decisions in ways similar to humans. As a vast field of research, AI encompasses various subfields, including Machine Learning and Deep Learning, which are closely related and interconnected with areas like Data Science. The figure below shows the interconnection between these 3 fields with Data Science:

Within AI, several core topics include problem-solving using search techniques, knowledge and reasoning (such as resolution, knowledge representation, and fuzzy logic), adversarial search for game playing (using algorithms like Minimax and Alpha-Beta Pruning), and managing uncertain knowledge with probabilities and Bayesian Networks. Additionally, AI is applied in creating expert systems.[1]

This paper explores problem-solving approaches in AI, specifically focusing on uninformed and informed search methods. Emphasis is placed on key algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), A*, and Best-First Search, providing insights into their structure and applications. [2]

Search algorithms form the backbone of AI pathfinding, widely applicable in robotics, video games, and logistics. Traditionally, search algorithms vary in complexity and are chosen based on the specific requirements of the application—such as memory constraints, time sensitivity, and accuracy.

The following are the two different types of search techniques under which all the above 6 algorithms come under:

Uninformed Search:

Uninformed search, also known as blind or brute force search, involves search techniques like Breadth-First Search, Depth-First Search, Uniform Cost Search, Iterative Deepening, and Bidirectional Search. This type of search lacks any information about the steps needed to reach the goal from the current state. [1] Rather than prioritizing the most efficient path, uninformed search pursues the path that appears promising at each step without considering its overall optimality.[9]

Informed Search:

Informed search, or heuristic search, uses a heuristic function to estimate the proximity of a given state to the goal. This approach, which follows a generate-and-test strategy, includes algorithms such as hill climbing, steepest ascent, A*, AO*, and Best-First Search. Informed search is generally more efficient than uninformed search due to its use of a heuristic function to guide the search toward the goal. Unlike uninformed search, informed search evaluates each possible next node using an evaluation function, or heuristic, to identify the most promising path from the current state to the goal.[10]

□ WORKING OF SEARCH TECHNIQUES:

1. Breadth-First Search (BFS):

In Breadth-First Search, nodes are explored level by level, starting from the root and expanding each level's nodes before moving to the next. [2] This method ensures that nodes closest to the root are fully explored before reaching deeper levels, thereby systematically working toward the goal. BFS organizes the search frontier as a queue, adhering to a First In, First Out (FIFO) order. □Applicability of Breadth-First Search (BFS):

Breadth-First Search is particularly useful when:

- A solution is needed that minimizes the number of turns or steps.
- There are limited solutions, with at least one offering a short path.
- Memory constraints are not a concern.[11]

□Applications of BFS:

- Finding the minimum spanning tree and shortest paths in unweighted graphs.
- Analyzing shared networks.
- Structuring connections on social networking platforms.
- Navigation in GPS systems to determine optimal routes.
- Verifying whether a graph is bipartite. [1]

2. Depth-First Search (DFS):

Depth-First Search begins at the source node, exploring as far down a branch as possible before backtracking to search alternative paths toward the goal.[11] DFS uses a stack to manage its frontier, following a Last In, First Out (LIFO) approach. This makes DFS a recursive algorithm since the stack handles the sequence of operations. The time and space complexities of DFS are $O(bm)$ and $O(b^m)$, where b is the branching factor and m represents the maximum depth.

for Both BFS and DFS

□Applicability of Depth-First Search (DFS):

Depth-First Search is suitable in scenarios where:

- Memory resources are limited.
- The order of neighboring nodes can be adjusted in the stack to increase the likelihood of finding a solution on the first attempt.
- The search space is finite, as DFS can struggle with infinite graphs or cyclic paths, which may lead to getting trapped in loops. [2]

□Applications of DFS:

- Detecting cycles or loops within a graph.
- Identifying strongly connected components in a graph.
- Solving puzzles with a unique solution path.

3. Uniform Cost Search:

Uniform Cost Search expands nodes based on the lowest-cost path, ensuring that the least expensive route is always chosen first. UCS uses a priority queue to manage nodes, calculating each node's cost using the equation $c(m) = c(n) + c(n, m)$, where $c(n)$ is the cost of the path from the start node to node n , and $c(n, m)$ is the cost of the edge from node n to node m .

the cost of the current node, $c(n)$ is the cost of the previous node, and $c(n, m)$ represents the edge weight.[10]

If edge costs vary, Breadth-First Search can be extended to Uniform Cost Search (UCS). Rather than expanding nodes by their depth from the root, UCS expands nodes based on their cumulative cost from the root. At each step, UCS selects the next node nnn with the lowest path cost $g(n)$, where $g(n)$ represents the total cost of edges from the root to node nnn . Nodes are organized in a priority queue to ensure the lowest-cost node is expanded first. This method closely resembles Dijkstra's algorithm for finding the shortest path from a single source. [3]

□ Advantages of UCS:

Uniform Cost Search is effective in finding the path with the lowest cumulative cost in a weighted graph, where each edge has a distinct cost. By always expanding the least costly path at each state, UCS guarantees an optimal solution, ensuring the shortest path from the start node to the goal.

□ Disadvantages of UCS:

One downside is that maintaining the priority queue requires keeping the list sorted, which can be computationally expensive. Additionally, the memory requirements for UCS can be prohibitively large. Another challenge is preventing the algorithm from entering an infinite loop, as it evaluates all possible paths from the start node to the goal node. [3]

4. A* Search Algorithm:

The A* algorithm combines the strengths of Uniform Cost Search and pure heuristic search to efficiently find an optimal solution. To compute the cost of each node, A* uses the equation $f(m) = c(m) + h(m)$, where $c(m) = c(n) + c(n, m)$ is the actual cost from the start node to node m , and $h(m)$ is the heuristic function. The goal of A* is to minimize the total cost $f(m)$, choosing the node with the lowest f -value for expansion at each step. Common heuristic functions include Euclidean distance (for free movement in any direction), Manhattan distance (when movement is restricted to four directions), and Diagonal distance (for eight-direction movement, like the king in chess). If two nodes have the same f -value, the node with the lower heuristic value $h(m)$ is selected. The algorithm terminates once the goal node is selected for expansion. An admissible heuristic function ensures that the solution remains optimal by never overestimating the cost (i.e., $h(n) \leq h^*(n)$). This allows for revisiting nodes from the closed list to the open list if necessary to find the optimal path. A*'s time and space complexities are $O(b^d)$, where b is the branching factor and d is the solution depth.

The steps to reach the goal from the source node are as follows:

1. Begin at the source node and add it to the open list.
2. Explore all neighboring nodes adjacent to the current node in the open list.
3. Calculate the cost for each discovered node and place them in the open list in increasing order of cost.
4. Move the current working node from the open list to the closed list.
5. The node at the front of the open list becomes the current working node.

6.Repeat steps 2-5 until the current working node is the goal node. [4]

7.The closed list contains the shortest path, and the final cost function value provides the optimal cost.

5. Best First Search(Greedy Search):

Best-First Search combines elements of both Breadth-First Search and Depth-First Search. It is implemented using a priority queue to select the most promising node for expansion at each step. The advantage of Depth-First Search lies in its ability to find a solution without exploring all nodes, while Breadth-First Search guarantees a solution without the risk of getting stuck. Best-First Search merges these benefits, allowing for flexibility in path selection.[10] At each stage, it evaluates the nodes and chooses the most appropriate one for expansion, regardless of its depth level.[4] This allows it to switch between strategies resembling Depth-First and Breadth-First Search, making it a "greedy" search algorithm. Its time and space complexities are $O(bd)O(b^d)O(bd)$, where b is the branching factor and d is the solution depth.

□Advantages of Best-First Search:

Best-First Search always selects the most promising path based on the current evaluation, making it effective in scenarios where the immediate best path needs to be prioritized. It is most beneficial in best-case scenarios where this approach leads directly to an optimal solution.

□Disadvantages of Best-First Search:

Being a recursive algorithm, Best-First Search is susceptible to getting stuck in an infinite loop if not carefully implemented. One significant limitation is its short-sightedness; it focuses on maximizing immediate gains without considering long-term outcomes, which may result in suboptimal solutions.[7]

□Applications & Limitations of Best-First Search:

Best-First Search is useful when:

- The search space is finite, as it can, like Depth-First Search, be made complete. However, it is unreliable because:
- In the worst case, its time complexity is equivalent to that of Depth-First Search.
- It is not guaranteed to produce an optimal solution.

This study explores five prominent AI search algorithms along with one hybrid search algorithm: Depth-First Search (DFS), Breadth-First Search (BFS), Uniform Cost Search (UCS), A* Search, Greedy Best-First Search (Greedy BFS), and a hybrid BFS-A* approach. The goal is to evaluate each algorithm's effectiveness in maze navigation by analyzing their performance across varying maze sizes. [5]

We focus on four key performance metrics: time complexity, space complexity, path length, and the number of explored nodes. These metrics are measured for mazes of different dimensions (10x10, 20x20, 30x30, 40x40, and 50x50), allowing for a comprehensive assessment of each algorithm's scalability and efficiency.

By examining the algorithms under consistent conditions, the study seeks to determine which search techniques offer the best performance in maze traversal, considering factors such as memory usage, execution speed, and the ability to find the shortest path.

Literature Review:

1. In "Comparative Analysis of Search Algorithms" the paper by Ronit Patel and Maharshi Pathak explores two types of search algorithms in AI: uninformed and informed. Uninformed techniques like Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform Cost Search (UCS) operate without guidance towards a goal, relying on systematic or cost-based exploration. BFS works level-by-level, useful for shortest paths but memory-intensive, while DFS explores deeply and suits memory-limited tasks despite possible infinite loops. UCS, optimal for weighted paths, can be time-consuming. Informed methods, such as A* and Best First Search, utilize heuristics to efficiently direct searches. A* balances path cost and heuristic estimates, excelling in navigation and gaming, whereas Best First Search provides faster, though sometimes suboptimal, results by alternating depth and breadth strategies. The paper concludes with a comparative analysis of these algorithms' time and space efficiency, favoring heuristic approaches for optimal and practical solutions.
2. The paper "Comparative Analysis of Search Algorithms in AI" by Fatima Sohail Shaukat, Ayesha Shafique and Ayesha Islam Qadri categorizes AI search algorithms into uninformed and informed types. Uninformed methods, like Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform Cost Search (UCS), operate without specific guidance, exploring nodes systematically or based on path cost. BFS and DFS have distinct strengths in different scenarios, with BFS ensuring optimal solutions in unweighted graphs, while DFS is better for memory-limited tasks. Informed algorithms, such as A*, Greedy Best First Search, and Beam Search, incorporate heuristic functions to guide the search, enhancing efficiency and relevance for complex tasks like pathfinding and AI game applications. The paper evaluates these algorithms based on completeness, optimality, time complexity, and space complexity, ultimately advocating for heuristic methods due to their practicality and speed in large problem spaces.
3. The paper "Comparative Study of Various Searching Algorithms" by Deepika Garg explores AI search algorithms, distinguishing between uninformed and informed search methods. Uninformed methods like Breadth-First Search (BFS) and Depth-First Search (DFS) operate without guidance toward the goal, with BFS suitable for situations without memory constraints and DFS for scenarios where multiple paths exist. Informed search, including A* and AO* algorithms, employs heuristics to prioritize promising paths, thereby increasing efficiency in reaching goals. The paper also highlights fuzzy logic as a tool for AI, particularly useful in modeling uncertainty and non-linear problem spaces, with applications in fields like robotics and gaming. The study concludes that heuristic-driven algorithms and fuzzy logic greatly enhance AI problem-solving capabilities.
4. The research paper "Comparative Analysis of Various Uninformed Searching Algorithms in AI" by Gayathri R. presents a comparative study of several uninformed search algorithms, specifically Breadth First Search (BFS), Depth First Search (DFS), Iterative Deepening Search (IDS), Uniform Cost Search (UCS), and Depth Limit Search (DLS). The author examines each algorithm's performance in terms of time complexity, space complexity, optimality, and completeness within artificial intelligence applications, particularly in solving shortest path problems. Findings indicate that Uniform Cost Search is the most effective in achieving optimal solutions, while Depth Limit Search and Depth First Search are less reliable due to their limitations in completeness and memory usage. The study concludes that UCS, BFS, and IDS are preferred for applications requiring optimal solutions, as they consistently expand nodes at the shallowest level, enhancing efficiency and effectiveness in goal-reaching scenarios.

5. The research paper "Comparison Analysis of Breadth First Search and Depth Limited Search Algorithms in Sudoku Game" by Tirsia Ninia Lina and Matheus Supriyanto Rumetna investigates the application of Breadth First Search (BFS) and Depth Limited Search (DLS) algorithms to solve Sudoku puzzles. This study explores how these algorithms handle the systematic search needed to fill the puzzle grid without violating Sudoku's constraints. While BFS systematically explores all possible number placements, making it thorough but memory-intensive, DLS limits the depth of search, thus using less memory and offering faster solutions in cases with constrained depths. The findings reveal that DLS is more efficient and faster in most scenarios, although BFS's structured approach can uncover multiple solutions, which can be beneficial when a puzzle has more than one possible answer. The paper suggests that for standard 3x3 Sudoku grids, DLS is generally more practical, but further studies could apply these algorithms to larger grid sizes or hybrid approaches for enhanced performance.

6. The thesis "A Comprehensive Comparative Study among Important Heuristic Search Algorithms in Artificial Intelligence" by Loai Riad Nawaf Al-Qudah offers an in-depth comparison of various search algorithms, including both blind search methods (Depth First Search and Breadth First Search) and heuristic approaches (Best First Search, Hill-Climbing, and A*). The study focuses on evaluating these algorithms based on criteria such as time complexity, search space complexity, optimality, and completeness, using the 8-puzzle problem as a testing scenario. Through experiments involving solution metrics like node generation, solution length, and running time, the study highlights the distinctions and efficiencies of heuristic searches over blind methods. Ultimately, it provides a recommendation framework indicating the suitability of each algorithm under different conditions, underscoring A* as a generally preferred heuristic for optimal solutions due to its balanced performance across various scenarios.

7. The paper, "Search Algorithms Under Different Kinds of Heuristics—A Comparative Study" by A. Bagchi and A. Mahanti, investigates three heuristic search algorithms, namely Algorithms A, B, and C, focusing on their performance under relaxed admissibility conditions. The study evaluates these algorithms by comparing node expansion count and solution cost, especially considering two types of heuristic estimates: proper and path-dependent. The authors find that Algorithm C consistently outperforms both A and B, providing a minimal-cost solution in cases with proper heuristics and requiring fewer node expansions. Algorithm B, a modified version of Nilsson's algorithm, is shown to have fewer expansions than A but still falls short of C. The paper highlights that C offers significant advantages in cases with path-dependent heuristics, reinforcing its suitability for complex heuristic search scenarios.

8. The paper "Comparative Analysis of Ant Colony and Particle Swarm Optimization Techniques" by V. Selvi and Dr. R. Umarani presents a comparative study of two prominent swarm intelligence optimization techniques: Ant Colony Optimization (ACO) and Particle Swarm Optimization (PSO). The study explores their applications, advantages, and limitations in solving complex optimization problems inspired by biological swarm behaviors, such as those observed in ants and bird flocks. ACO is based on the pheromone trail-laying behavior of ants and is effective for discrete combinatorial problems like the traveling salesman problem, where it adapts well to dynamic conditions. Conversely, PSO draws inspiration from social behaviors and efficiently tackles continuous optimization problems by guiding particles through the search space based on both individual and neighboring particles' best positions. The authors highlight that while ACO is advantageous in path-finding applications, PSO is suited to scenarios requiring adaptability and multidimensional search. Both algorithms show potential for enhancements through fitness sharing and adaptive methods for broader applications in dynamic, real-time, and multi-objective optimization contexts.

Existing Methodology:

1. Puzzle Solving (e.g., Sudoku, N-Puzzle):

- Researchers often use structured puzzles like Sudoku or the N-Puzzle, which are ideal for testing search algorithm performance under fixed conditions. In Sudoku, Depth-Limited Search (DLS) and BFS might be evaluated based on the time to find a solution or the algorithm's handling of large solution spaces. For N-Puzzle, the focus is on comparing how algorithms reach the goal state (solved puzzle), with metrics like number of moves, search depth, and time taken, revealing each algorithm's efficiency, backtracking, and completeness in constrained environments. [5]

2. Game AI (e.g., Tic-Tac-Toe, Chess):

- Search algorithms are widely used in game AI to simulate potential moves and predict outcomes. Researchers might use DFS or BFS variations like Minimax or Alpha-Beta pruning to test algorithm efficiency and accuracy in achieving winning moves or avoiding losses.[6] For games with higher complexity (like chess), these algorithms are tested based on computational limits, search depth, and decision-making speed. Studies focus on metrics such as branching factor, depth of exploration, and nodes pruned (in alpha-beta pruning), which indicate each algorithm's computational feasibility for real-time strategy games.

3. Real-Time Applications (e.g., Robot Navigation):

- In robotic navigation, pathfinding algorithms are evaluated based on how quickly they can identify feasible routes and adjust to dynamic changes, such as newly introduced obstacles. Researchers often test A*, Greedy BFS, and UCS in real-time simulations, measuring the speed and accuracy of pathfinding under varying obstacle densities and time constraints. These studies provide insights into an algorithm's robustness, adaptability to changes, and efficiency in practical, real-time applications.

4. Hybrid and Heuristic Variants:

- Studies on hybrid models and heuristic-enhanced algorithms involve combining features from different search algorithms to improve performance.[7] For example, a BFS-A* hybrid could be designed to prioritize nodes closer to the goal while maintaining a breadth-first approach to ensure solution completeness. Researchers compare these hybrid approaches to traditional algorithms using metrics like search depth, computational time, and solution optimality, often demonstrating that hybrids can offer improved performance in specific cases (e.g., large grids or complex, high-cost environments).

The Following are more of the existing methodologies related to AI Search algorithms:

- 1.Route Optimization (e.g., Traveling Salesman Problem)
- 2.Scheduling Problems (e.g., Job Scheduling)
- 3.Natural Language Processing (e.g., Parsing and Tokenization)
- 4.Decision-Making Systems (e.g., Multi-Agent Pathfinding)
- 5.Network Optimization (e.g., Shortest Path in Graphs)
- 6.Resource Allocation (e.g., Load Balancing)
- 7.Constraint Satisfaction Problems (e.g., CSPs in AI Planning)

These methodologies allow researchers to test AI search algorithms in controlled, application-specific scenarios, providing a solid foundation for understanding algorithm efficiency, scalability, and applicability in various real-world problems.[8]

Proposed methodology:

Comparing and Analyzing different Search algorithms inside a Maze environment of different sizes (Grid-Based Pathfinding):

This research employs a Python-based simulation to evaluate six algorithms under uniform maze conditions of 10x10, 20x20, 30x30, 40x40 and 50x50 configurations.

1.Algorithm Design: Each algorithm is implemented with standardized start and goal locations within a maze. The hybrid BFS-A* method incorporates an adaptive threshold for heuristic adjustment, switching between BFS and A* based on the distance from the goal.

2.Maze Structure: The maze is designed with fixed and random obstacles, maintaining consistency across algorithms and tests. Each algorithm runs independently, and a record of time, memory, and path details is maintained.

3.Performance Metrics: The study measures time complexity, space complexity, path length, and explored paths, providing quantifiable insights into each algorithm's performance across increasing maze sizes.

4.Data Analysis: Results are averaged across maze configurations to capture each algorithm's average performance, highlighting trade-offs in time, memory, and path efficiency.

Scope of the work:

This study's findings are anticipated to contribute valuable insights into the practical application of AI search algorithms in structured and constrained environments.

1.Algorithmic Efficiency: By analyzing standard and hybrid search algorithms in a consistent setting, this study sheds light on each algorithm's real-world applicability based on efficiency and resource management.

2.Hybrid Algorithm Viability: Testing the BFS-A* hybrid model offers insights into its feasibility for scenarios where both speed and optimal pathfinding are essential.

3.Implications for AI Development: These results are significant for developing AI systems in robotics, autonomous navigation, and gaming where maze-like, grid-based environments are common. Identifying resource-efficient algorithms could directly impact the scalability and cost of AI applications.

Implementation in Detail:

Implementation Process:**1.Initial Setup:**

oAfter running the code, users must:

- ☐Select the start goal and the end goal by clicking on the grid.
- ☐Choose the algorithm they want to visualize.
- ☐Press the spacebar to visualize the algorithm in real time.

oThe width and height of the grid maze are configurable in the code, allowing customization.

oThe number of rows and columns in the maze can also be adjusted to suit specific visualization needs.

Visualizing the Algorithm:

2.A `time.sleep(0.02)` delay is added during the visualization process for better clarity and understanding of how the algorithm explores the maze.

3.The code is divided into three parts because of the lengthy code. The first part contains codes for BFS, DFS and UCS; the second part contains codes for A* and Greedy BFS and the third part contains code for the hybrid algorithm made using Breadth first search and A*.

Node Selection:**4.Users can manually select the start and end nodes using the mouse:**

- o Start Node: Highlighted in red.
- o End Node: Highlighted in green.

5.This contains convenient keys to make the different functions of the maze work for example:

- (S) to regenerate the maze
- (Spacebar) to start the visualisation
- In BFS, DFS, UCS - press 1 to switch into BFS mode, press 2 to switch into DFS mode, And press 3 to switch into UCS mode.
- In A* and Greedy BFS – press 1 to switch into A* and press 2 to switch into Greedy BFS.
- We can choose the number of rows and columns we want for the maze to increase or decrease the complexity and size of the maze.

Pathfinding and Display:

- If a path is found, it is displayed visually on the grid.
- If no path is found, a “Path Not Found” message appears on the screen for 2 seconds.



6. Once the maze traversal for the algorithm is finished and the optimal path/solution is found the different parameters like are show in the output window of the application:

- Execution time
- Maximum memory usage
- Nodes explored
- Path length

For Hybrid Algorithm (BFS+ A*):

o For shorter paths, the system uses BFS for simplicity.

o If the threshold exceeds a predefined limit (indicating a longer path), the system dynamically switches to A* for better performance.

7. This insights can be used to compare these algorithms to see which one performs better with less amount of time consuming less amount of space and traversing the least amount of nodes to reach the Goal node.

8. Thus through empirical testing and visualization of results, this research contributes to a deeper understanding of algorithm selection criteria, guiding practitioners in choosing the most suitable search strategy for specific AI application. Through empirical testing and visualization of results, this research contributes to a deeper understanding of algorithm selection criteria, guiding practitioners in choosing the most suitable search strategy for specific AI application.

Experimental setup and results:

In order to Analyze these search algorithms by comparing them with each other in a Maze environment of different sizes we need to:

(Steps:)

1. Create a virtual maze in python language using libraries like:

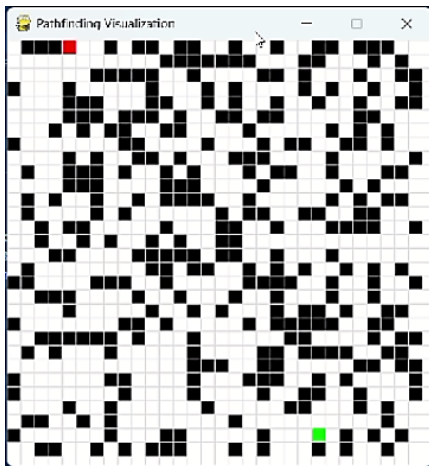
- pygame: Used for creating 2D games and multimedia applications.
- deque: Provides a double-ended queue for efficient insertion and deletion at both ends.
- heapq: Implements a heap queue algorithm, often used for priority queues.
- time: Used to measure the time performance of the algorithms.
- random: Used for wall generation.
- math: Provides mathematical functions like trigonometry, logarithms, and constants.
- tracemalloc: Tracks memory allocation in Python, useful for memory profiling.

- `pygame.freetype`: Renders text with custom fonts in pygame applications.

2. As seen above using the various libraries provided by the python language we will start creating the walls(black blocks) and the paths(white blocks) inside the maze of different sizes namely (10x10 to 50x50), various colors like green for the actual path, blue for all the nodes traversed etc. were declared at the start of the code as shown below:

```
# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
GREEN = (0, 255, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
GRAY = (220, 220, 220)
WALL = (50, 50, 50)
YELLOW = (255, 255, 0) # Highlight for start/goal
```

3. At the beginning of running the code we are asked to choose a starting (Red block) and ending (Green block) node so that the algorithms can traverse through the maze as shown below:



4. Then by pressing the “Space bar” button to start making the algorithm traverse through the maze as shown in the code.

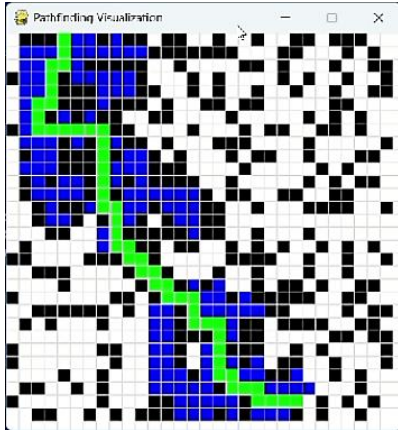
```
# Press 'SPACE' to Start the Selected Algorithm
elif event.key == pygame.K_SPACE and start and goal and algorithm:
    elapsed_time, memory_usage, nodes_explored, path_length = algorithm(start, goal, grid)
    results.append((elapsed_time, memory_usage, nodes_explored, path_length))
    start = None # Reset start for the next run
    goal = None # Reset goal for the next run
    algorithm_name = ""
```

6. The traversal (path or nodes explored) of the algorithm to find the goal node is shown using blue blocks, so that we can visualize the paths explored by each algorithm to reach the Goal node.

```
# Visualize BFS progress
neighbor.color = BLUE
draw(WIN, grid)

# Mark the final path in green
for node in path:
    node.color = GREEN
draw(WIN, grid)
```

Once the traversing of the algorithm through the maze is finished a green path is highlighted which is supposedly the shortest path found possible by that algorithm.



Once the algorithm has traversed through the maze and found the shortest(most optimal) path then the “Execution time, Maximum memory usage, Nodes explored and Path length” are displayed in the output terminal of the Application using which we can compare each algorithm to make educated insights about each algorithm as to which algorithm is suitable for which particular tasks.

```
PS C:\Users\vaishali\OneDrive\Desktop\Comparative
pygame 2.5.2 (SDL 2.28.3, Python 3.10.9)
Hello from the pygame community. https://www.pyg
A* Algorithm selected.
Execution Time: 0.6352 seconds
Max Memory Usage: 28
Nodes Explored: 65
Path Length: 29
```

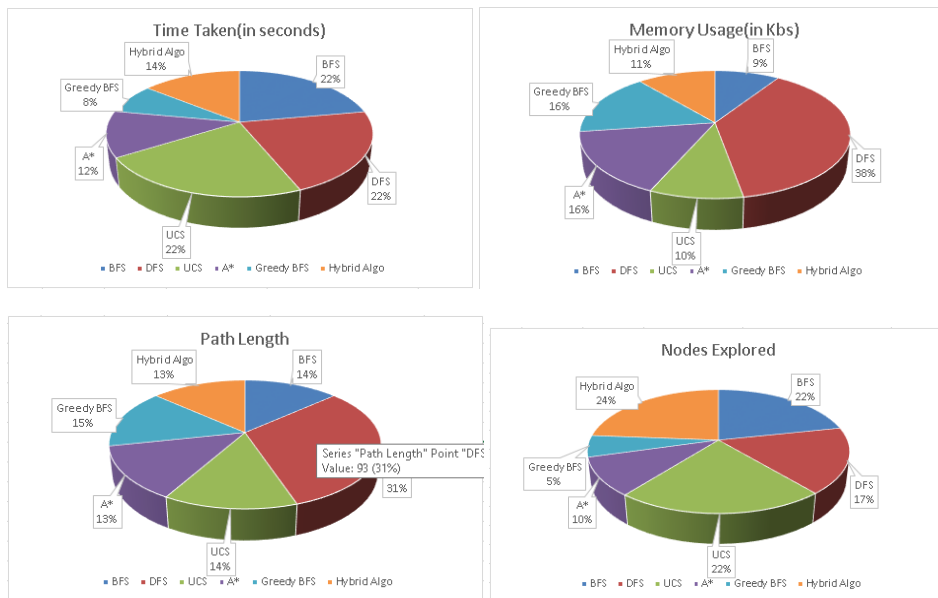
Analysis of the result:

After running the code for all the Algorithms through the maze we get several results for different sizes of maze like for every algorithm i.e. (BFS, DFS, UCS, A* and Greedy BFS and Hybrid Algo) we get 5 results for the different parameters namely: “Maze size, Time taken, Memory usage, Path length and Nodes explored”; since there are 5 different sizes of Mazes(10x10, 20x20, 30x30, 40x40, 50x50)So with all these values we get 30 rows of information for the above parameters and an average of those values are calculated to see which algorithm is more optimal in finding the bestest path possible without consuming much resources.The table for the above is given below with concise details along with its Bar Graph and Pie chart for better visualization.

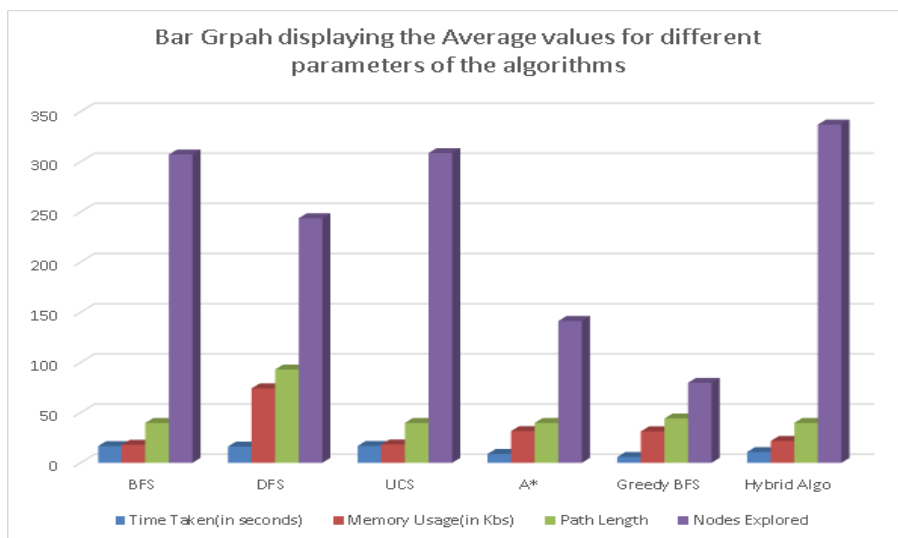
Algorithm	Time Taken(in seconds)	Memory Usage(in Kbs)	Path Length	Nodes Explored
BFS	16.71536	18	39.8	307.4
DFS	16.22162	74.2	93	243.8
UCS	16.84804	18.4	39.8	308.8
A*	8.89452	31.6	39.8	141.4
Greedy BFS	5.86566	31.4	44.2	79.8
Hybrid Algo	10.77494	21.8	39.8	337.2

Maze Size	Time Taken(in seconds)	Memory Usage(in Kbs)	Path Length	Nodes Explored
10x10	2.6463	9	15	50
20x20	3.3636	6	24	60
30x30	16.0046	20	48	299
40x40	20.3946	20	37	378
50x50	41.1671	35	75	750
10x10	1.4869	10	15	22
20x20	3.0792	13	32	42
30x30	9.5481	51	66	145
40x40	14.5527	75	95	201
50x50	52.4412	222	257	809
10x10	2.6419	10	15	50
20x20	3.365	6	24	60
30x30	15.9759	21	48	298
40x40	20.55	20	37	379
50x50	41.7074	35	75	757
10x10	1.9106	9	15	29
20x20	2.4818	12	24	33
30x30	8.6888	39	48	126
40x40	9.1779	57	37	120
50x50	22.2135	41	75	399
10x10	2.3856	8	17	39
20x20	2.4859	20	24	25
30x30	7.4439	39	56	102
40x40	7.1312	41	41	97
50x50	9.8817	49	83	136
10x10	0.2324	3	15	7
20x20	1.2304	5	24	38
30x30	8	15	48	257
40x40	17.6789	29	37	575
50x50	27.2244	57	75	809

❖ Pie Chart for Visualization:



❖ Bar Graph for Visualization:



Conclusion:

This research successfully analyses the comparative performance of six AI search algorithms—BFS, DFS, UCS, A*, Greedy BFS, and a hybrid BFS-A*—in solving pathfinding problems within maze environments of varying complexities. Each algorithm demonstrated unique strengths and weaknesses when evaluated on key metrics such as time complexity, space complexity, path length, and the number of explored nodes.[4]

BFS and DFS excelled in scenarios where memory constraints or exploration of all potential solutions were critical, although DFS struggled with optimality in complex mazes. UCS emerged as a reliable choice for finding cost-optimal paths in weighted graphs, albeit with significant memory overhead. In contrast, A* leveraged heuristic guidance to achieve optimal results efficiently, making it ideal for scenarios requiring both speed and accuracy. Greedy BFS, while faster, often fell short of optimal solutions due to its short-sighted heuristic approach. The hybrid BFS-A* algorithm showcased a balanced performance by combining the memory efficiency of BFS with the pathfinding precision of A*.

The visualization of results provided insights into the real-world applicability of these algorithms. Smaller mazes revealed BFS's exhaustive yet thorough nature, while larger and more complex mazes highlighted A*'s efficiency in balancing exploration and heuristic-driven directionality. The hybrid algorithm, though computationally demanding, exhibited promise for applications requiring both scalability and optimality.[9]

This study underscores the importance of selecting appropriate algorithms tailored to specific problem constraints. By demonstrating the trade-offs between resource consumption and accuracy, the research provides a foundational guide for deploying these algorithms in practical AI applications such as robotics, game development, and autonomous navigation systems

Future Enhancement:

1. **Dynamic Maze Environments:** Incorporating dynamic obstacles or real-time changes to the maze could simulate more realistic scenarios, such as robotic navigation in unpredictable environments. Algorithms could be evaluated on their adaptability and resilience to changes.
2. **Multi-Agent Pathfinding:** Extending the study to include multi-agent scenarios, where multiple entities simultaneously navigate the maze, could test algorithms for collision avoidance and cooperative strategies.
3. **Integration of Advanced Heuristics:** Exploring advanced heuristic functions tailored to specific applications, such as terrain-based costs in robotics or game-specific heuristics, could improve the accuracy and efficiency of algorithms like A* and Greedy BFS.
4. **Hybrid Algorithm Optimization:** Further refinement of the hybrid BFS-A* approach, including adaptive thresholds or learning-based heuristic adjustments, could enhance its scalability and applicability to larger and more complex mazes.
5. **Real-World Applications:** Testing algorithms in real-world scenarios, such as autonomous vehicle navigation or robotic pathfinding in industrial settings, would provide valuable insights into their practical viability.

6. Comparison with Swarm Intelligence Techniques: Future studies could compare classical search algorithms with swarm intelligence methods like Ant Colony Optimization (ACO) or Particle Swarm Optimization (PSO), analyzing their relative performance in maze environments.

References:

- [1] R. Patel, Charotar University of Science and Technology, M. J. Pathak, Lakehead University Thunder Bay Campus, and S. P. Rami, "Comparative analysis of search algorithms," Jun. 2018.
- [2] F. Sohail Shaukat, A. Shafique, and A. Islam Qadri, "Comparative analysis of search Algorithms in AI," Research, Oct. 2022. doi: 10.13140/RG.2.2.29282.61123.
- [3] D. Garg and The Technological Institute of Textile & Sciences Bhiwani, India, "COMPARATIVE STUDY OF VARIOUS SEARCHING ALGORITHMS," Mar. 2015.
- [4] C. Wilt, J. Thayer, W. Ruml, Department of Computer Science, and University of New Hampshire, "A comparison of greedy search algorithms," 2010.
- [5] T. N. Lina and M. S. Rumetna, "Comparison analysis of breadth first search and depth limited search algorithms in sudoku game," *Bulletin of Computer Science and Electrical Engineering*, vol. 2, no. 2, pp. 74–83, Dec. 2021, doi: 10.25008/bcsee.v2i2.1146.
- [6] L. R. N. Al-Qudah, "A comprehensive comparative study among important heuristic search algorithms in artificial intelligence," thesis, 2015.
- [7] A.B. Bagchi, A. M. Mahanti, and Indian Institute of Management Calcutta, "Search Algorithms Under Different Kinds of Heuristics--A Comparative study," Jan. 1983.
- [8] V. Selvi and R. Umarani, "Comparative analysis of ant colony and particle swarm optimization techniques," *International Journal of Computer Applications*, vol. 5–5, no. 4, Aug. 2010.
- [9] N. M. Sabri, M. Puteh, and M. R. Mahmood, "A review of gravitational search algorithm," *International Journal of Advances in Soft Computing and its Applications*, vol. 5, no. 3, pp. 1–28, Nov. 2013. [Online]. Available: [ResearchGate](#)
- [10] R. Gayathri, "Comparative Analysis of Various Uninformed Searching Algorithms in AI," *International Journal of Computer Science and Mobile Computing*, vol. 8, no. 6, pp. 57–65, June 2019. [Online]. Available: www.ijcsmc.com
- [11] Z. Bu and R. E. Korf, "Iterative-Deepening Uniform-Cost Heuristic Search," in *Proceedings of the Fifteenth International Symposium on Combinatorial Search (SoCS 2022)*, Los Angeles, CA, USA, 2022, pp. 20–28. [Online]. Available: www.aaai.org

